University of Bologna
Dipartimento di Informatica – Scienza e Ingegneria (DISI)
Engineering Bologna Campus

Class of

Infrastructures for Cloud Computing and Big Data M

Class Starting…

Basics, Objectives, and initial Models

Antonio Corradi, Andrea Sabbioni

Academic year 2025/2026

CLASS WEB SITE

Find

- Teaching contents (lessons, and more…)
- Information & discussion exchange
- Some project topic and area proposals
- **Cases for practice** available when you are
- **Middleware tools there**, also individual practice CORBA, OpenStack, Hadoop, SPARK, Kafka, Docker, Orchestrators, …

Via Web

- Many papers available
- Some personal deepening hints

virtuale.unibo.it

https://virtuale.unibo.it/course/view.php?id=35258

Mobile Middleware Research group

CLASS ASSESSMENT

The **final grading** stems from **an oral exam**
to ascertain the **knowledge** and **orientation** about the entire discipline, ranging on all topics, starting with the basics, going through the practical portions of middleware, and also with a possible follow-up on a chosen topic
Also, it is possible deepening on a specific agreed practical topic with some added (microprojects)

You can also choose the project activity (for 3 credits), recommended for the Distributed System Computer Engineering path
Assignment of a project on a specific **subject assigned for anyone** and **done individually**
A Report and a Presentation for the exam

PROJECT ACTIVITIES

Projects can deal with any topics of the class for 3 credits
And **Big Data** and **Cloud continuum** areas of **industrial interests**

- **Smart Cities**
- **Smart Tourism**
- **Industry 5.0**
- **Critical Infrastructures**
- **Cloud continuum Observability**
- **QoS-aware Observability for critical systems**
- **Cloud continuum Network management**
- **Cloud Infrastructure for IoT provisioning**
- **QoS-aware microservice composition in Service Meshes**
- **Serverless Storage**
- **Hardware acceleration in Service Mesh**
- **…**

Class Main Goals

The course aims at delivering a novel vision of all distributed systems and at building a **deep, informal, practical, and meditated understanding** and **experience** of their **operations**

- **We are immersed into a pervasive ecosystem of distributed IT systems: personally, socially, and as part of organizations**

We are interested in **building a view** based on **a system perspective**, i.e., **what is behind those systems,** and their **behavior** and **impact,** from two perspectives

First the user viewpoint, but **more important** with **the perspective of implementers and designers**

The class focus on the **experience of long-term operations in system execution** rather than in static planning and configuration

We **aim at the entire life cycle, so focusing on the execution time operations**

COURSE TARGETS

Distributed Systems are available everywhere

We have to consider two perspectives for those systems in normal environments:

- **Big data available to anyone**
- **Many processing localities available over data**
- **Many applications of large dimensions to support**

In general, both companies and private people have reached homogeneity in IT resource usage

So, we have the same requirements and tools for both

- Companies have a *conservative attitude* toward ICT resources, but have also a **consolidated usage of *not on-premises resources***

COURSE TARGETS

Large global corporations that provide **Cloud services (Amazon, Google, IBM,…)**

Organization of internal architecture that provides Cloud services with needed Quality of Service

- **Cloud Data Center Organization**
- **Interaction with other Data Centers and Cloud**
- **Intra and inter Cloud**

In general, one **Cloud provider** has several local data centers and keeps them as a **central bone**, but has to maintain also ***external available resources* and *extra-organization agreement*** for special dedicated situations

FROM STARTUPS, TO BUSINESSES, TO CLOUD PROVIDERS

Since cloud and distributed infrastructure are becoming more and more pervasive, this course can provide the background toward an added value in your future careers

The following **4 typical situations** you can encounter in your future that can raise some practical **questions** that you can encounter and understand within the course concepts:

- The **Green Startup**
- The **Bank Consulting**
- The **Industry Consulting**
- The **Cloud Provider**

THE GREEN STARTUP

Scenario:
You have founded a startup aiming to monitor the status of home gardens and plants by providing suggestions to customers on how to treat at best each plant
Examples of initial questions: You are creating the first prototype of the app and developing the suggestion algorithm

Q1: Do you have to buy your own infrastructure to host services?
Q2: Which model of Cloud Computing is better to choose?

Q2.1: If the startup succeeds … can the service scale easily?
Q2.2: Do you have always to pay for all needed services? Even first days of life of our application?

FINANCE CONSULTING

Scenario:

You work for a consulting company operating in the finance sector od fintech and you must optimize IT infrastructure and services of a famous National Bank

The IT infrastructure is complex with many infrastructure and services to support the core business of the bank

As a technical consultant, you are expected to solve specific vertical business problems, integrating with existing solutions already implemented

Q1: how can you maintain online the home banking application and keep the system in a consistent state while the infrastructures creates backups of data?

Q2: how can you access records in a huge dataset that the bank has stored in partitions ?

DOMOTIC SERVICE CONSULTING

Scenario:
You work for a famous electric appliances industry who wants to integrate Smart Home **IoT**s with their furniture. You are the *Solution Architect* of the company and your boss asks you to design the backend services to gather data, enable remote control of appliances, and integrate Smart Home control

Q1: The customer requires that the system to be **always** available to reach a high level of customer satisfaction, it has to work under failure conditions, and it have to be **always** consistent as customer pay a fee for each registered device. Is it possible?

Q2: Logs sent by devices are huge in volume, how can you collect them? Are all data of the same importance/value? Can the system tolerate variations in load?

CLOUD PROVIDER

Scenario:

You work for a cloud provider as an expert on cloud infrastructures. The company is investing to improve the services offered.

Task 1: They want to integrate IoT provisioning and management as part of the Infrastructure offered to customers

Q1: Is it feasible?

Task2: IoT devices must be constantly monitored

Q2: Is it better if the IoT device periodically sends its metrics or if the infrastructure pulls the metrics from device? How can one decide whether the device sent all its data?

Task3: An integration with smart home can cause millions of concurrent writings on IoT configuration

Q3: Is it a problem for our large scale infrastructure?

University of Bologna
Dipartimento di Informatica – Scienza e Ingegneria (DISI)
Engineering Bologna Campus

Class of
Infrastructures for Cloud Computing and Big Data M

Components, Microservices, Container

Antonio Corradi, Andrea Sabbioni
Academic Year 2024/2025

THE GREEN STARTUP

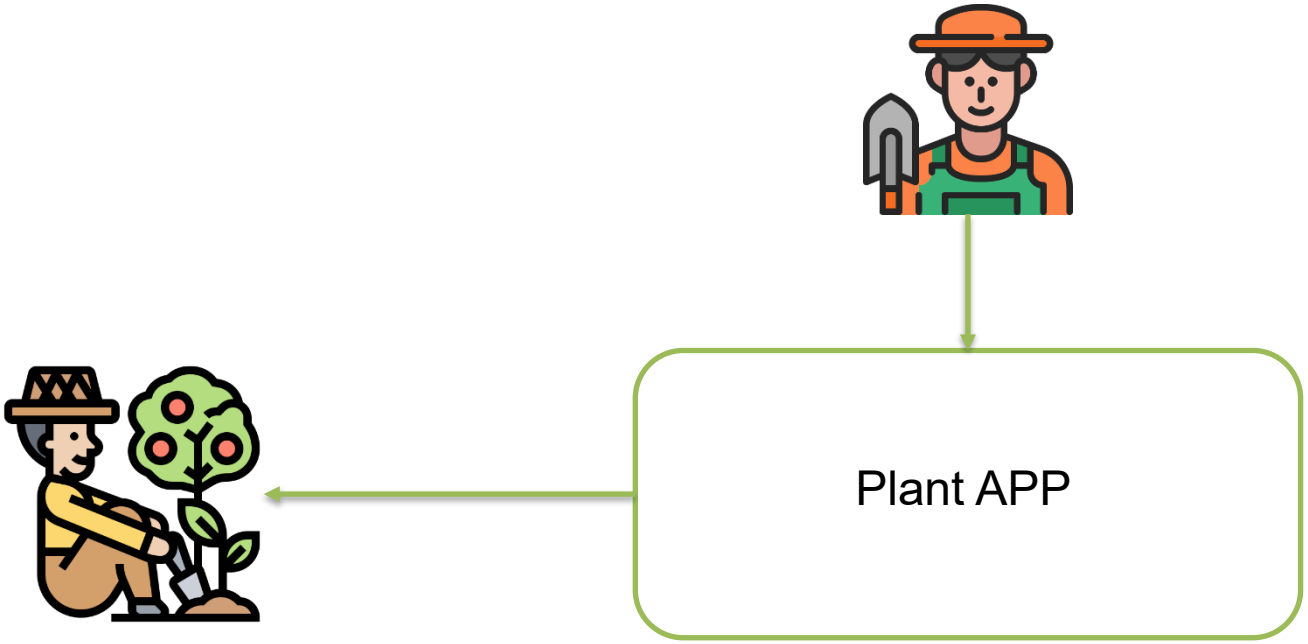
Scenario:
You have founded a startup aiming to monitor the status of home gardens and plants by providing suggestions to customers on how to treat at best each plant
Examples of initial questions: You are creating the first prototype of the app and developing the suggestion algorithm

Q1: Do you have to buy your own infrastructure to host services?
Q2: Which model of Cloud Computing is better to choose?

Q2.1: If the startup succeeds … can the service scale easily?
Q2.2: Do you have always to pay for all needed services? Even first days of life of our application?

Plant App

MVP: Create a service exposing a query service to retrieve plant information



Monolith

A Software Monolith has

- One code base
- One build and deployment unit
- One technology stack (Linux, JVM, Tomcat, Libraries)

Benefits

- Simple mental model for developer
- One unit of access for coding, building, and deploying
- Simple scaling model for operations
- Just run multiple copies behind a load balancer

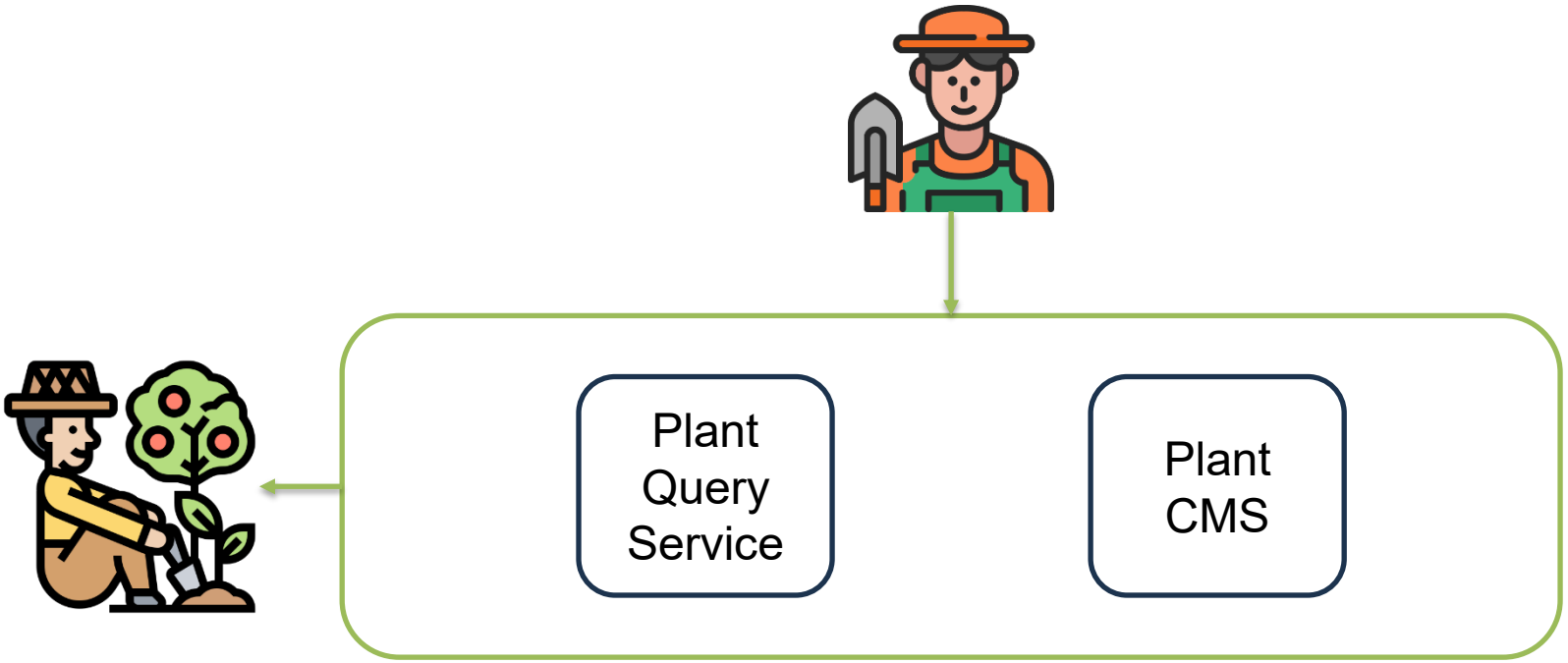
Monolith Issue

- Huge and intimidating code base for developers
- Development tools get overburdened
  - Refactoring take minutes
  - Builds take hours
  - Testing in continuous integration takes days
- Scaling is limited
  - Running a copy of the whole system is resource-intense
  - It doesn’t scale with the data volume out-of-the-box
- Deployment frequency is limited
  - Re-deploying means halting the whole system
  - Re-deployments will fail and increase the perceived risk of deployment

Plant App

We need to create two service:

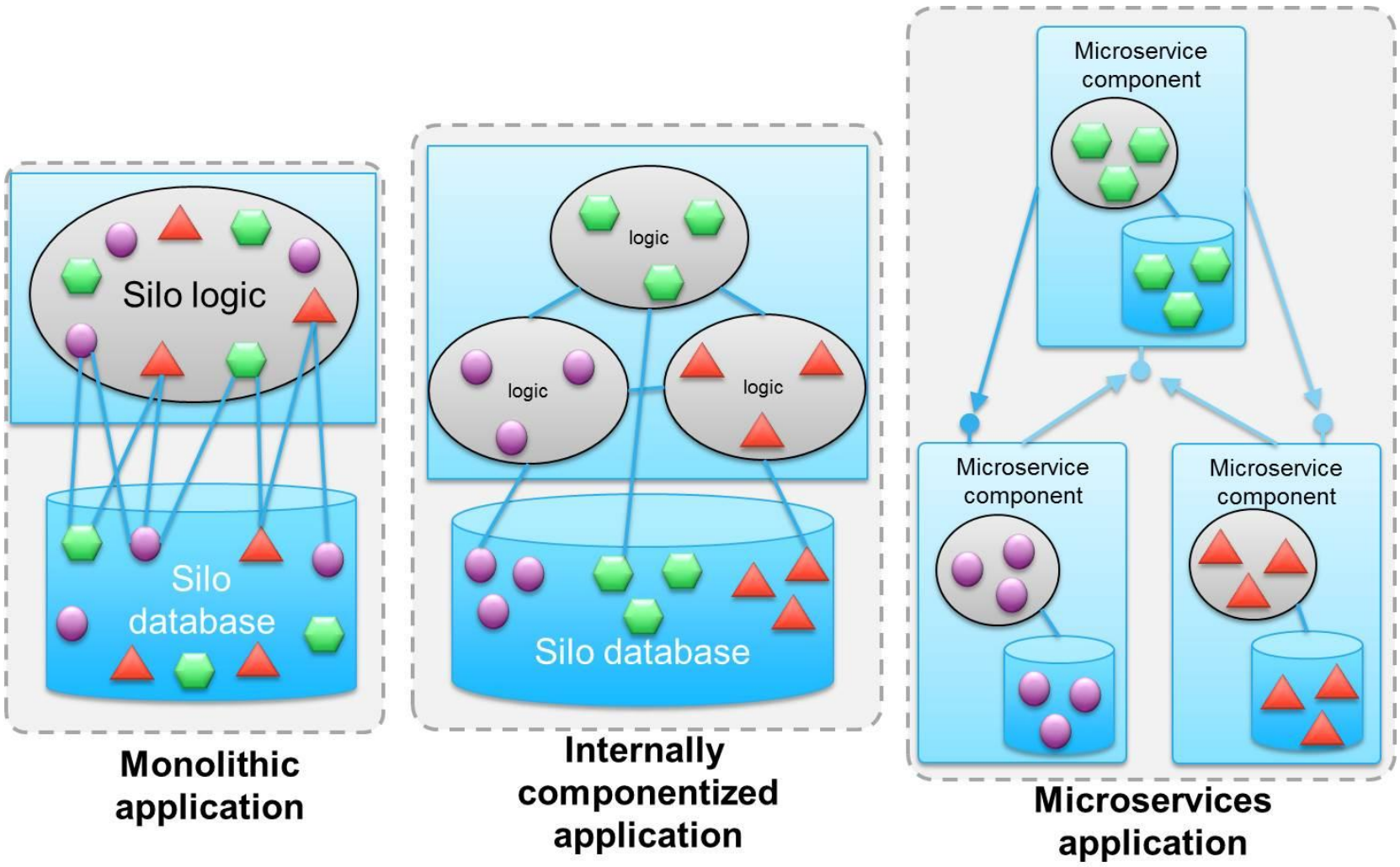
1. The Plant Content Management System service enables expert to populate plants and associate suggestions
2. The Plant Query Service enables end-users to submit query with different interfaces, such as natural language query, tag search, etc…



Microservice

In short, the microservice architectural style is an approach to developing a single application as a **suite of small services**, each **running in its own process** and communicating with lightweight mechanisms, often an HTTP resource API. These services are **built around business capabilities** and **independently deployable** by fully automated deployment machinery. There is a **bare minimum of centralized management** of these services, which may be written in different programming languages and use different data storage technologies.
-- James Lewis and Martin Fowler (2014)

Microservice



Microservice

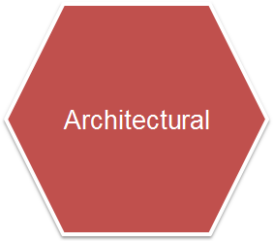
Loosely coupled Service Architecture with boundary context

On the logical level, microservice architectures are defined by a *functional system decomposition into* ***manageable*** *and* ***independently deployable components***

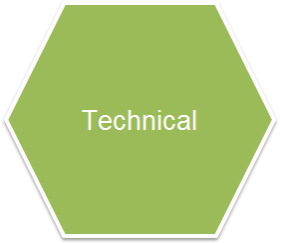
Microservices Architecture by Source A. Schroeder

- **Micro**, a microservice must be manageable by a single development team
- **Functional system decomposition** means vertical slicing
- **Independent deploy** no shared state and inter-process communication

Microservice



- Bounded Context
- Polyglot
- Single responsability
- Composable



- Run on its own process
- Isolates faults
- Deployed independently
- Scales independently
- Owns its data (**Why?**)

- Teams organized around business capabilities
- Culture of automation
- Small team
- *You build it you run it* … Wait … What??? Are you sure?
- Decentralized Governance

Microservices: independent unit of deploy

- Enables **separation** and **independent evolution** of code base
- (if needed) Independent technology stacks
- Ability to **scale only smaller portions** of system functionality
- Independent evolution of features

- Side effects
  - Multiple components -> more deploy units
  - More complexity
  - Automation is not an option

Microservices: Independent CodeBase

- Each service has its **own software repository**
- Codebase is maintainable for developers – **it fits into their brain**
- **Tools work fast** – building, testing, refactoring code takes seconds
- Scalable - Service startup only takes seconds
- No accidental cross-dependencies between code bases

Microserivces Independent Technology Stack

- Each service is implemented on its own technology stacks
- The technology stack can be selected to fit the task best
- Teams can also experiment with new technologies within a single microservice (smaller impact/risk, fosters applied R&D)
- No system-wide standardized technology stack also means
  - No struggle to get your technology introduced
  - No piggy-pack dependencies to unnecessary technologies or libraries

Microservices: Scaling

Independent ***business*** unit of deploy enable micro-services to scale best, fast and with less resource consumption

- Each microservice can be scaled independently
- Identified bottlenecks can be addressed directly
- Data sharding can be applied to microservices as needed
- Parts of the system that do not represent bottlenecks can remain simple and unscaled

Independent evolution of features

Microservices can be extended without affecting other services

- For example, **you can deploy a new version** of an application **without re-deploying the whole system**
- You can also go so far as to replace the service by a complete rewrite

You have to **ensure** that the **service interface remains stable**

Microservices Prerequisites

Before applying microservices, you should have in place

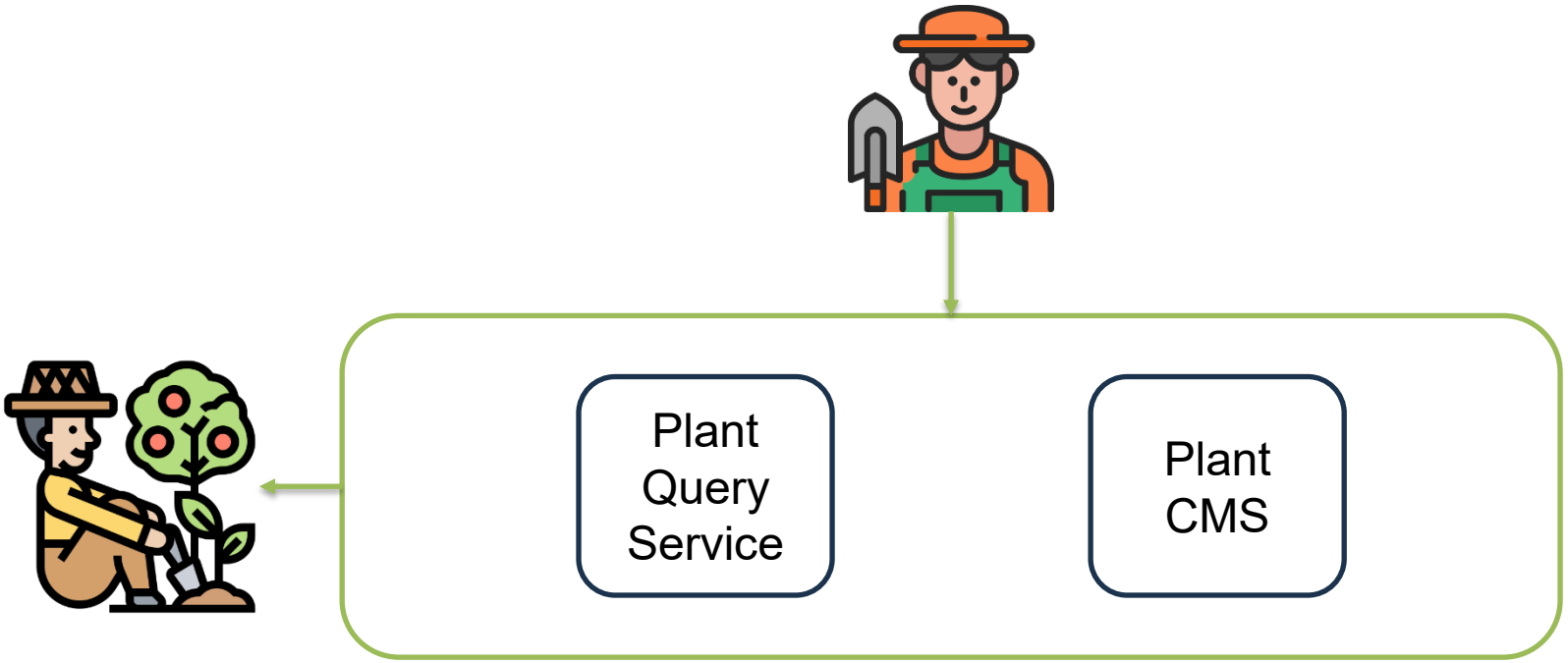
- **Rapid provisioning**
  - Dev teams should be able to automatically provision new infrastructure
- **Basic monitoring**
  - Essential to detect problems in the complex system landscape
- **Rapid application deployment**
  - Service deployments must be controlled and traceable
  - Rollbacks of deployments must be easy

Source http://martinfowler.com/bliki/MicroservicePrerequisites.html

Microservice Backend for Plant App

We need to create two service:

1. The Plant Content Management System service enables expert to populate plants and associate suggestions
2. The Plant Query Service enables end-users to submit query with different interfaces, such as natural language query, tag search, etc…



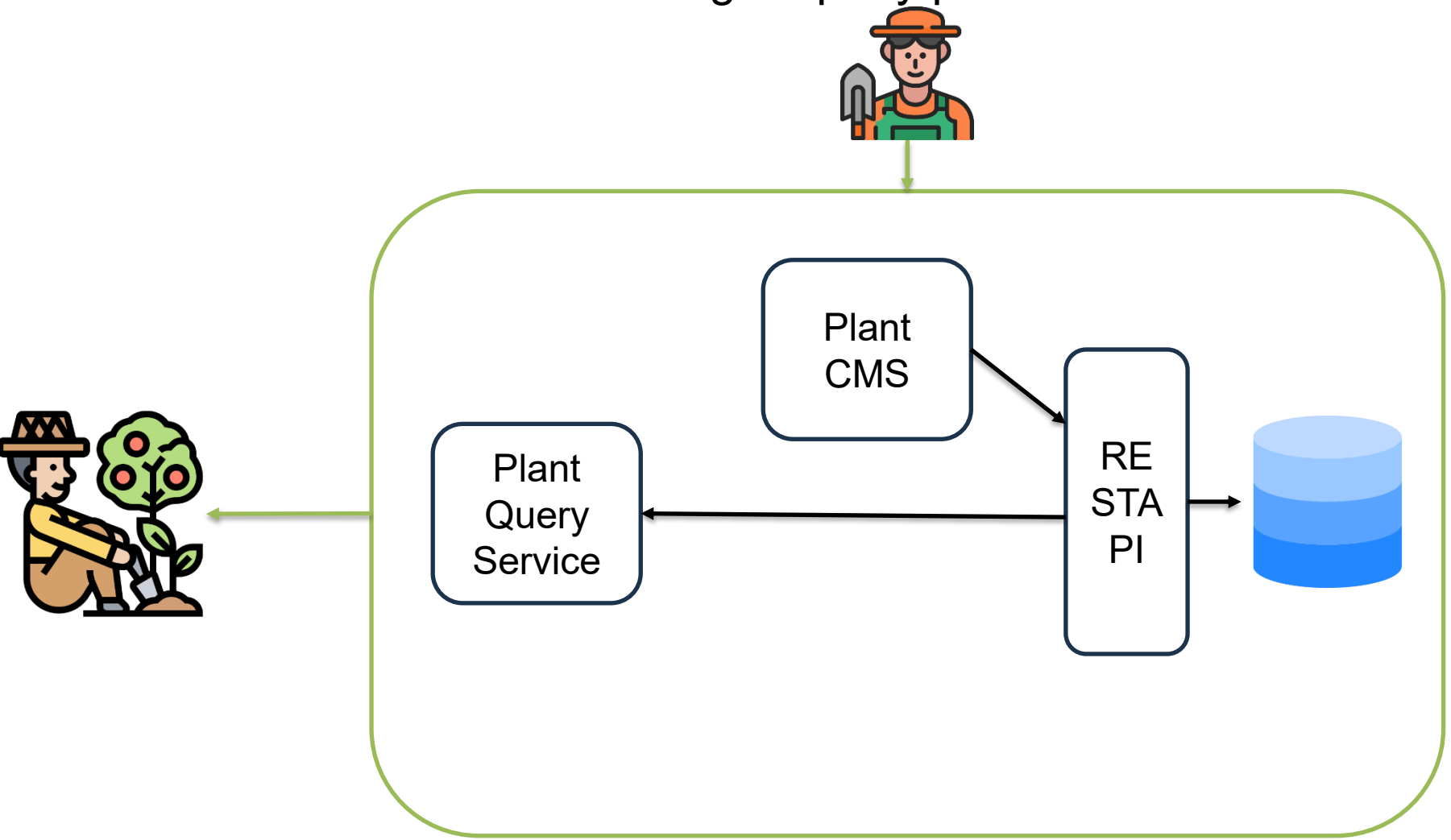
Plant App

Exercise 0: draw a small architecture for serving requests for the plant APP

?

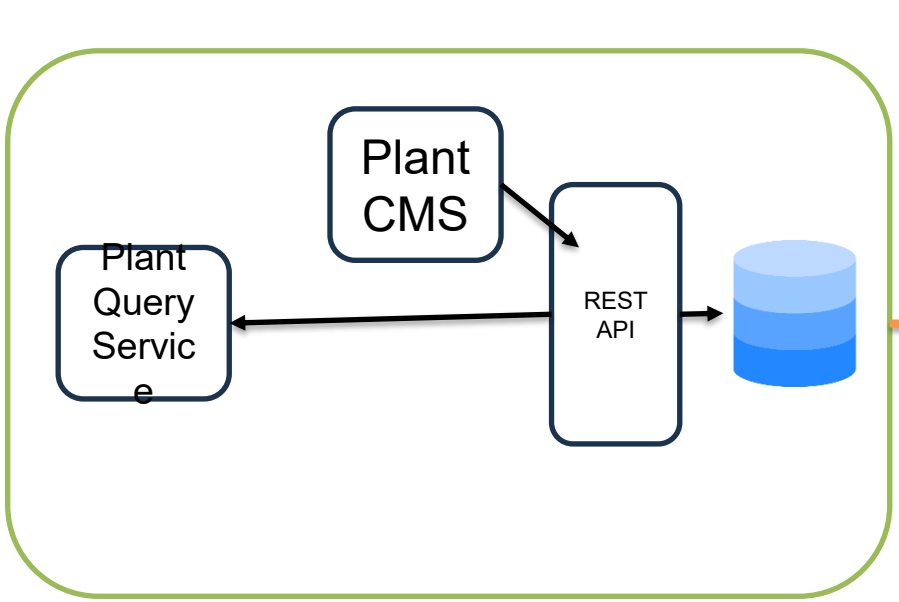
Microservice-based backend for Plant App

MVP: Create a service enabling to query plant information



Plant App Hosting

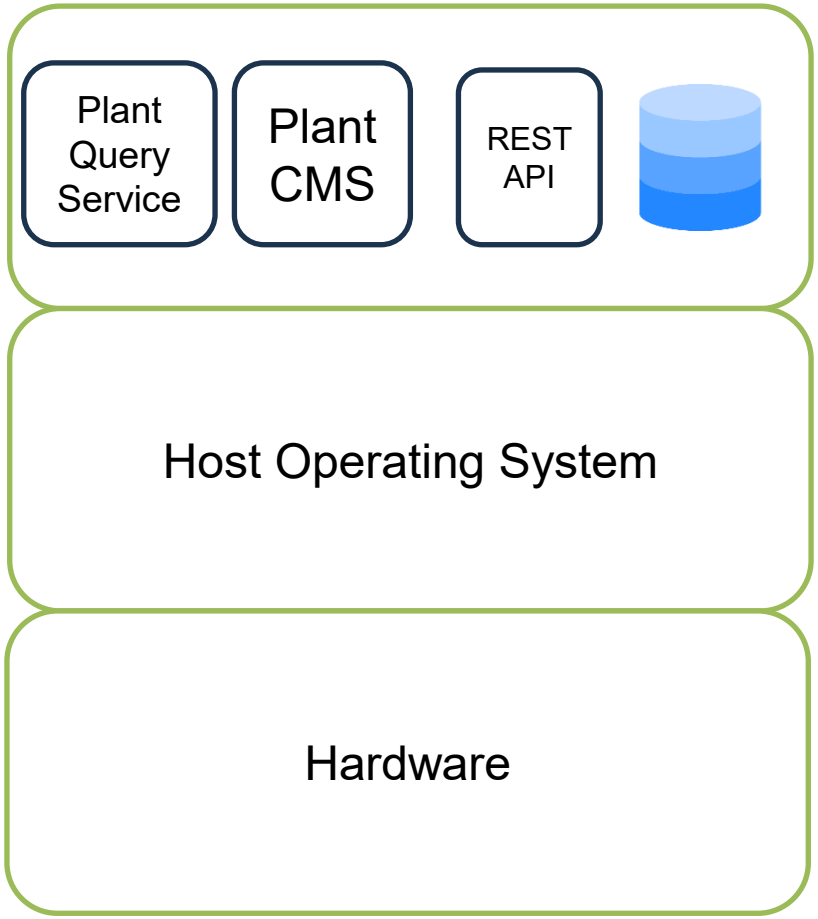
How you can **deliver**, **host**, and **manage** microservices of your application?



Microservice Hosting Bare Metal

Bare Metal:

- Can’t be shared
- Primitive management of services
- Security hazards
- Dependency management and conflicts resolution
- Possible conflicts among services
- Unpredictable resources allocation and deadlocks



Hosting Problems

Process Isolation

Process management

Process packing

Reproducible environment

Process Isolation

A process is the **representation of a running program** and consists of:

- Memory
- A set of data structures.

The kernel uses these data structures to store important information about the state of the program.

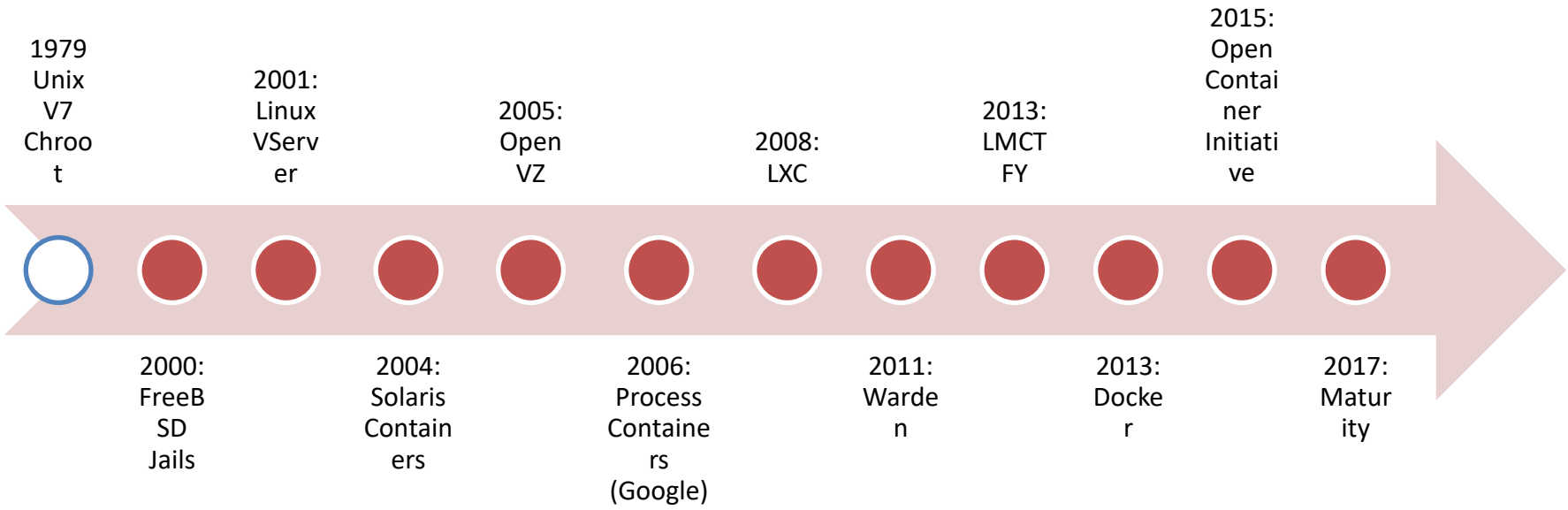
WHY?

- Security
- Interferences and deadlocks

UNIX systems isolate by default:

- Memory space.
- Privileges: A process gets the same privileges as the user that created the process.

Isolation



https://blog.aquasec.com/a-brief-history-of-containers-from-1970s-chroot-to-docker-2016

Containerized Process

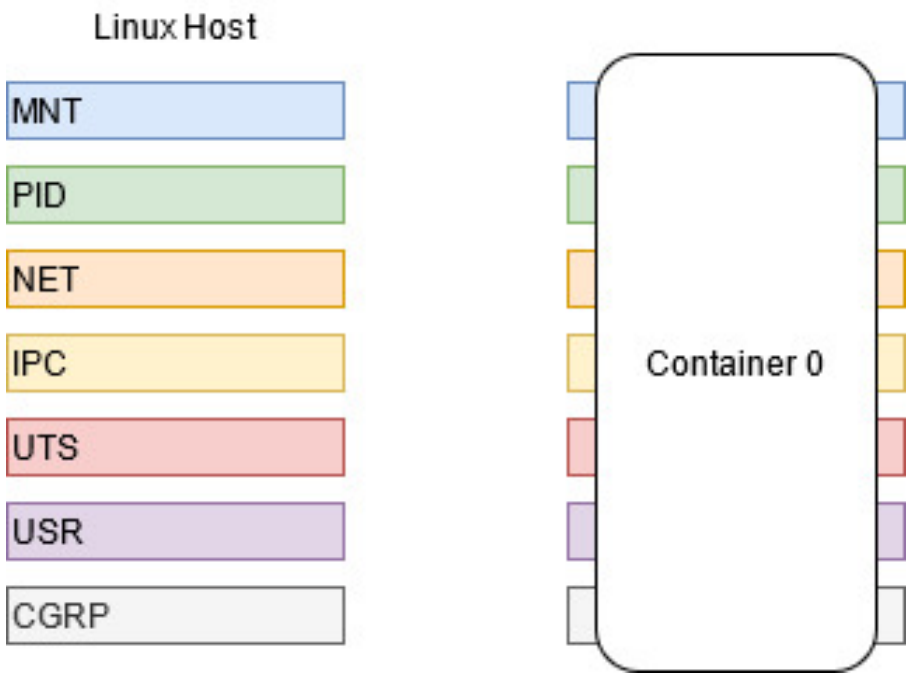
“It's just like any other process, except it has a very restricted view of the world. So, for example, you start a container. The process is given its own root directory, and it thinks that it's looking at the whole root directory of the whole computer, but it's actually only looking at a tiny subset of the file system.”

Liz Rice
Container Security Specialists
@Aqua Security

Linux Namespaces

Namespaces limit the scope of kernel-side names and data structures at **process granularity**

- **MNT**: Mount points
- **PID**: Process IDs
- **NET**: Network devices, stacks, ports…
- **IPC**: Message queues, semaphores and shared memory
- **UTS**: Hostnames and NIS Domain Name
- **USR**: User and group IDs
- **CGRP**: Quotas

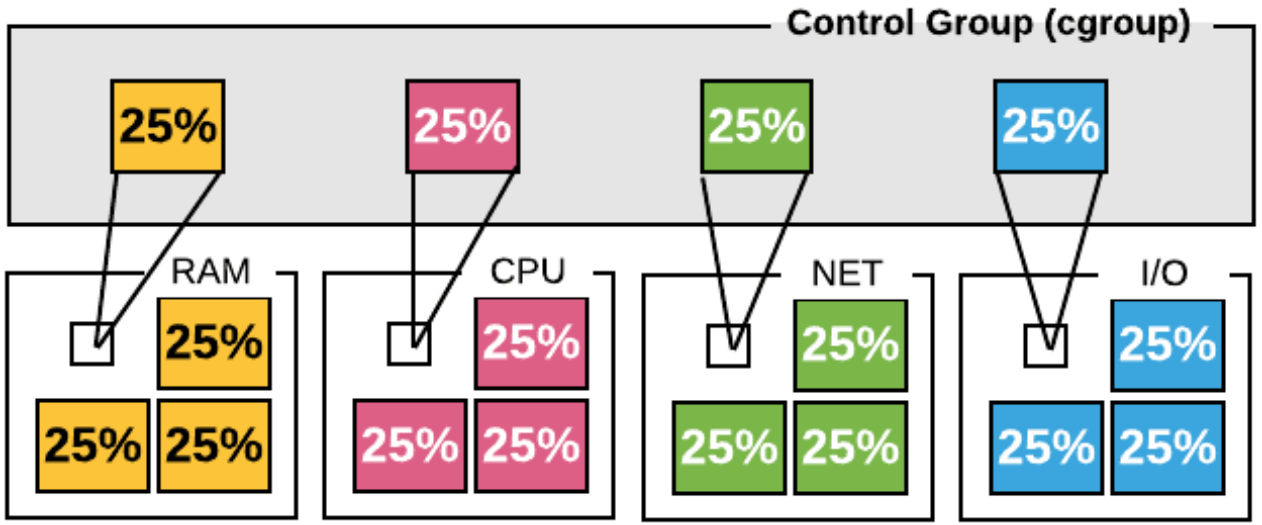


https://man7.org/linux/man-pages/man7/namespaces.7.html

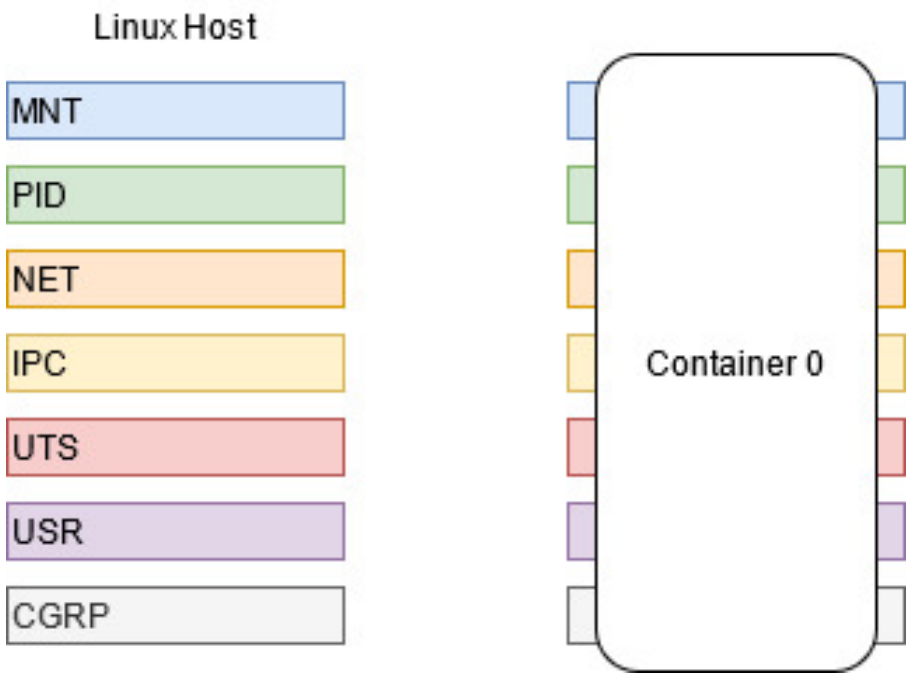
Containers C-Groups

C-Groups manages resources for groups of processes

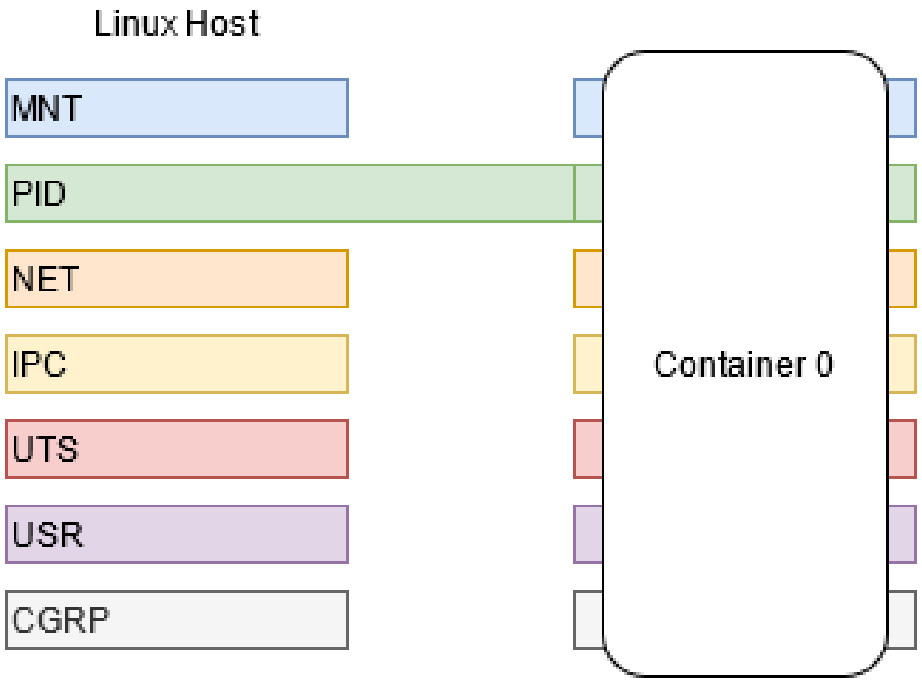
- **cpu controller**: by default, the kernel scheduler aims to give equal cpu time to all processes. cgroups can be used for fair grouping between arbitrary sets of processes (an example of 30 apache processes and 10 postgres processes)
- **net_cls** - interface for tagging network packets with a class identifier (so that you could later add rules based on packet class)
- **memory** controller has hierarchical support and allows for soft limits (cgroup can use as much memory as needed provided there is no memory contention and the hard limit is not exceeded).
  - Hierarchical support means that child cgroups contribute to the memory usage of their ancestors. If an ancestor exceeds a limit, memory will be reclaimed from the ancestor and all its children
- **cpuset** is also hierarchical



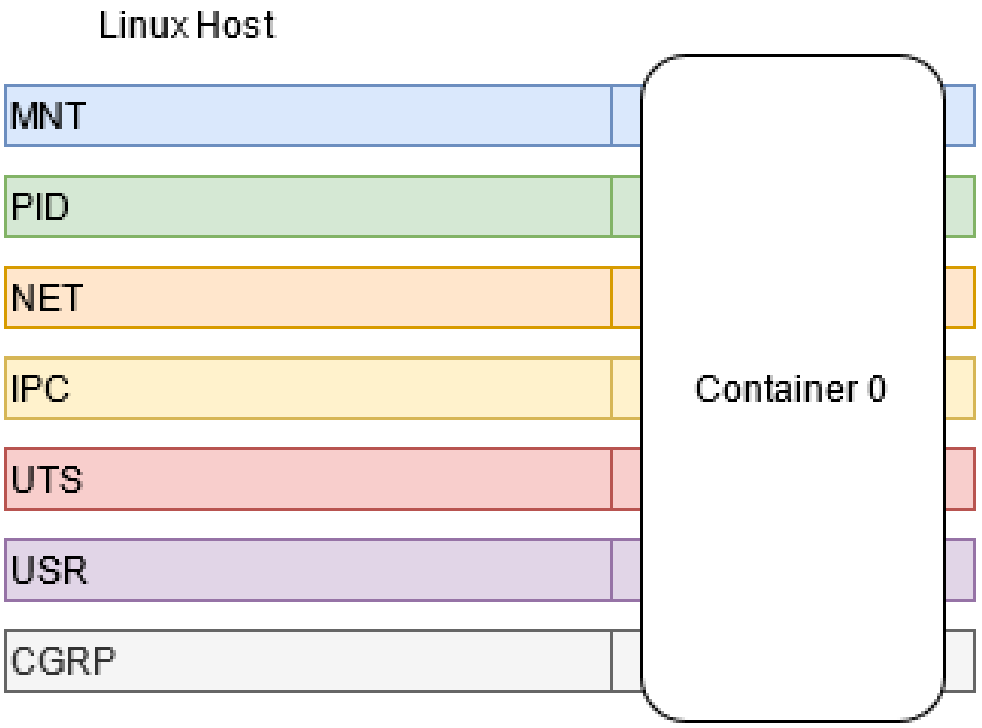
Linux Namespaces- Isolated



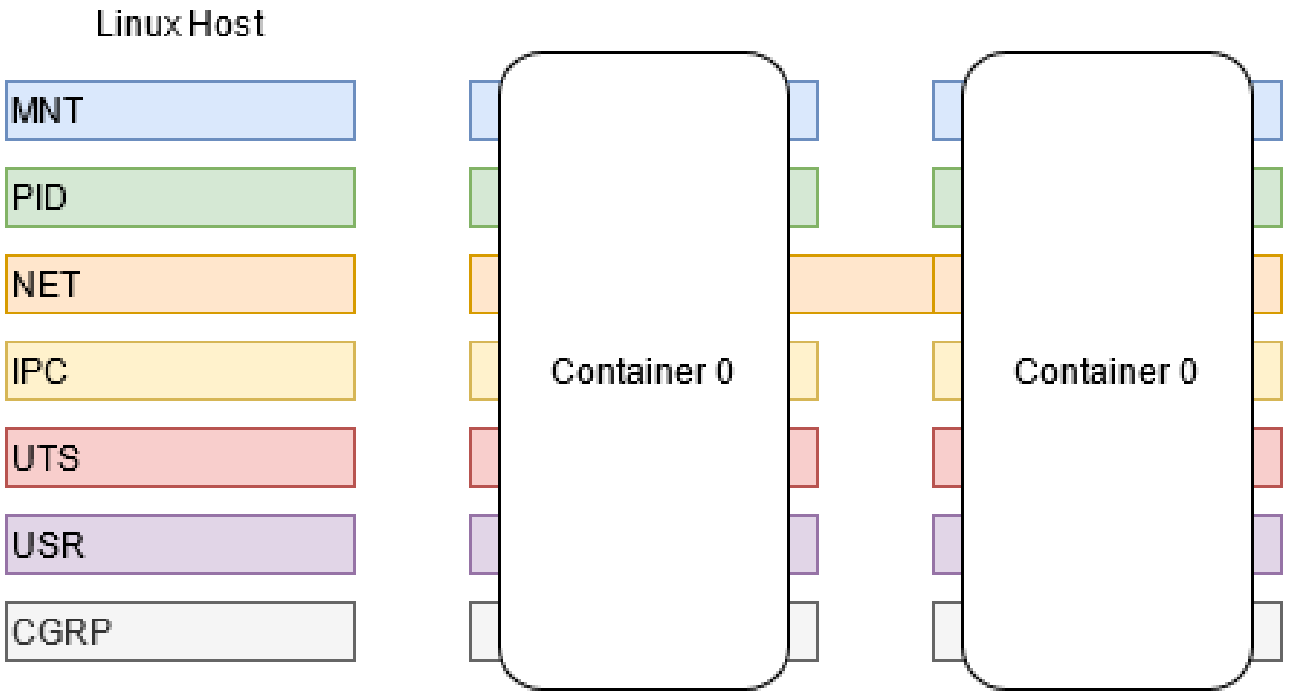
Linux Namespaces-1 Shared



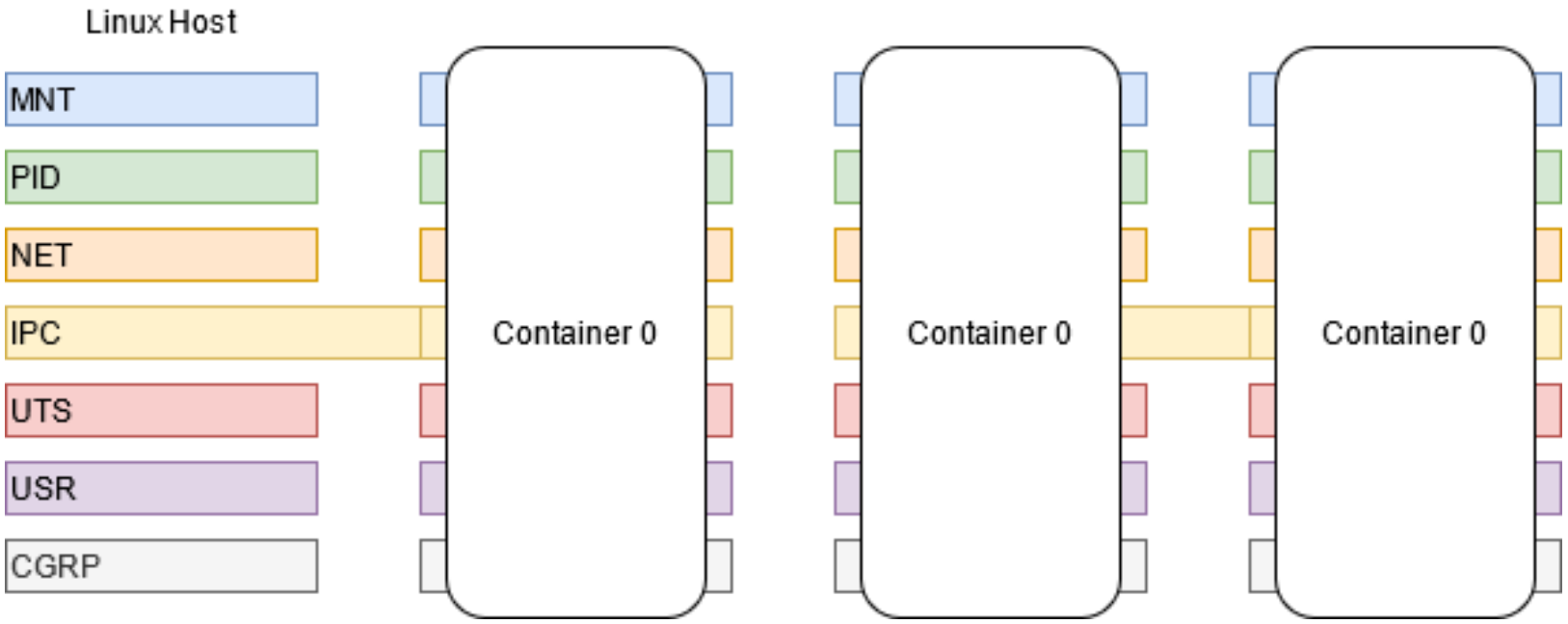
Linux Namespaces- All Shared



Linux Namespaces-Shared among Containers

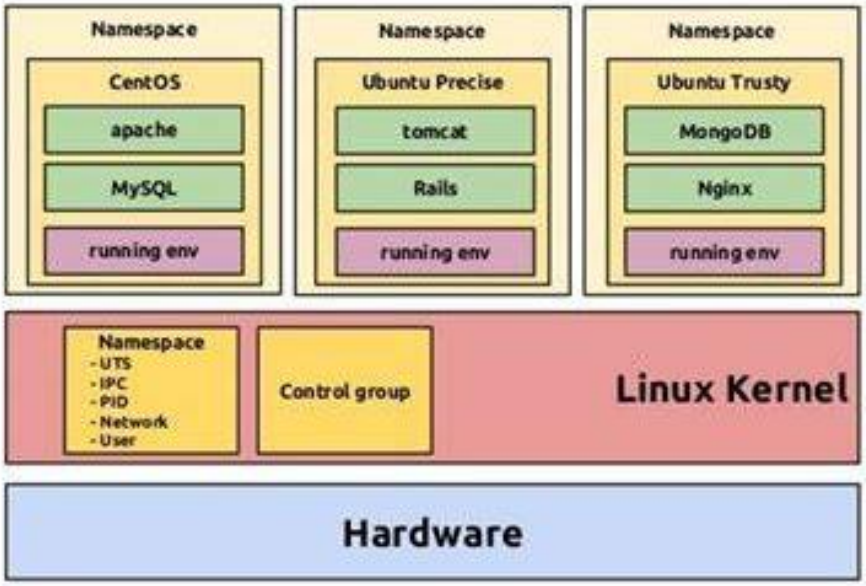


Inter Process Communication

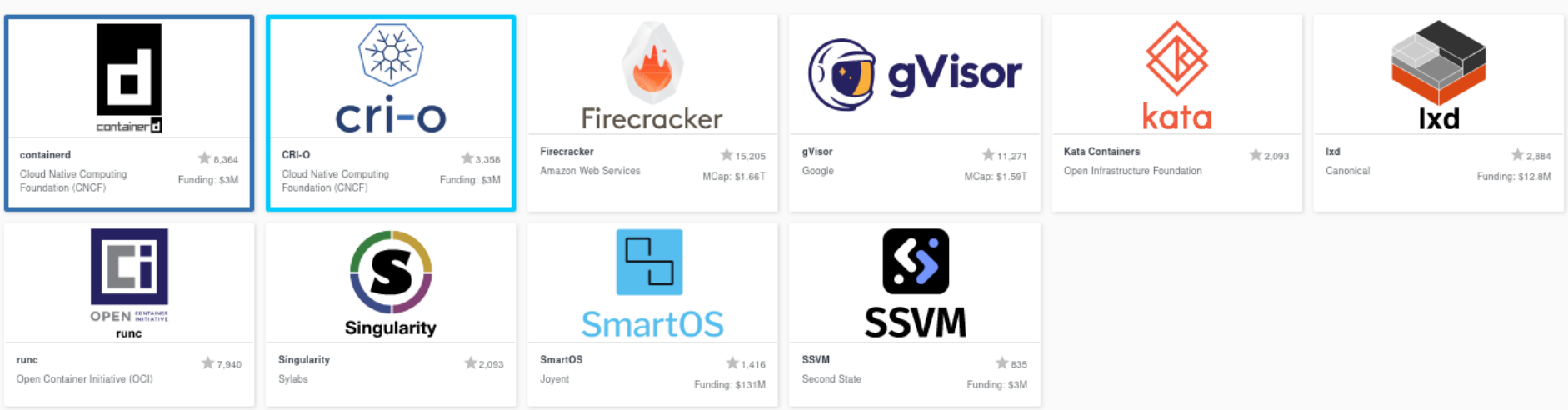


Containers

- A light form of resource virtualization based on kernel mechanisms
- A container is a *user-space* construct
- Multiple containers run on top of the same kernel
  - illusion that they are the only one using resources (cpu, memory, disk, network)

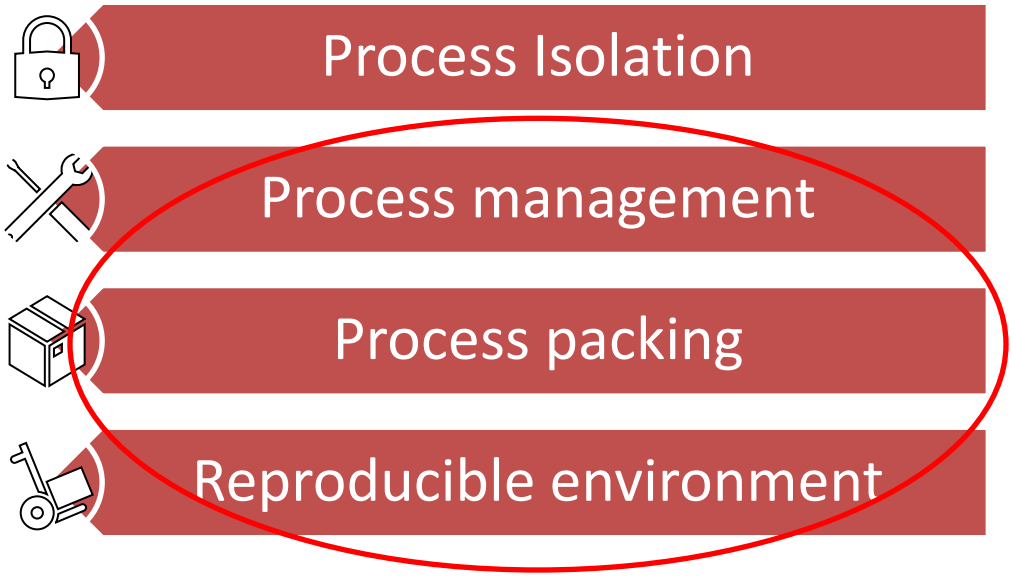


The Container Runtime Landscape

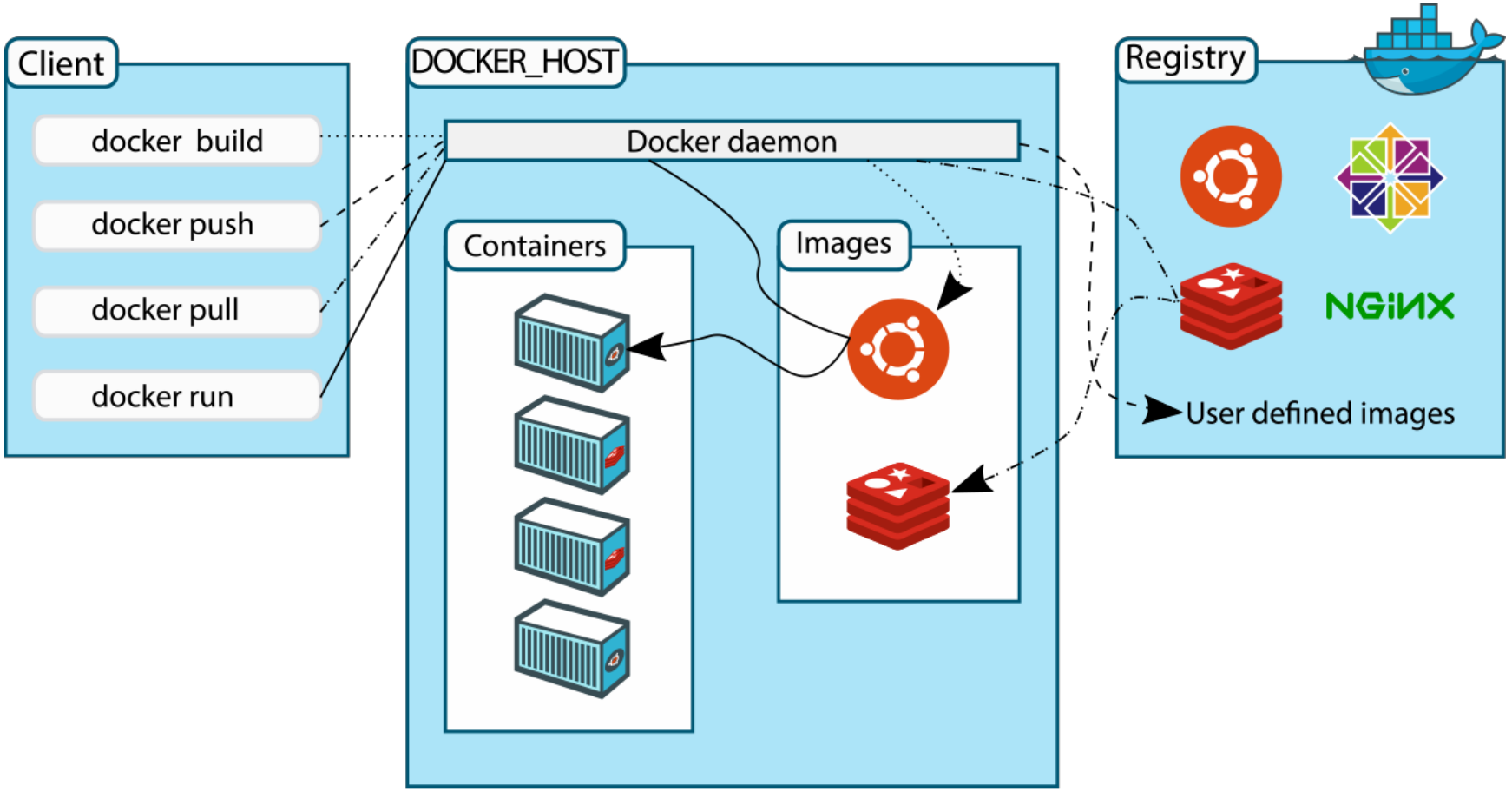


https://landscape.cncf.io/card-mode?category=container-runtime&grouping=category

Problems

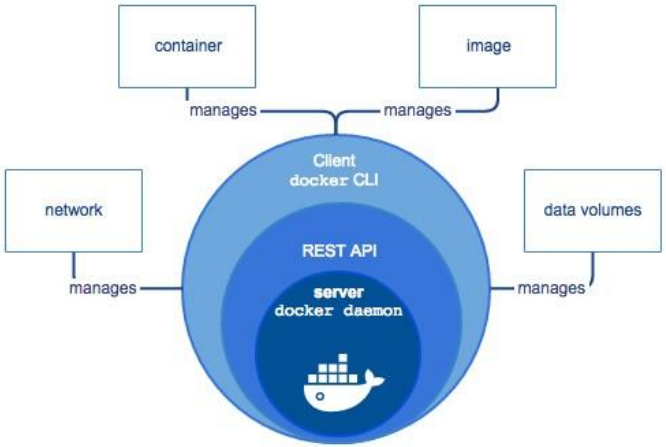

Process Isolation
Process management
Process packing
Reproducible environment

Docker Platform



Docker Engine

Docker is a container engine that enables **configuration**, **deployment**,
and **Lifecycle Management** of containers.

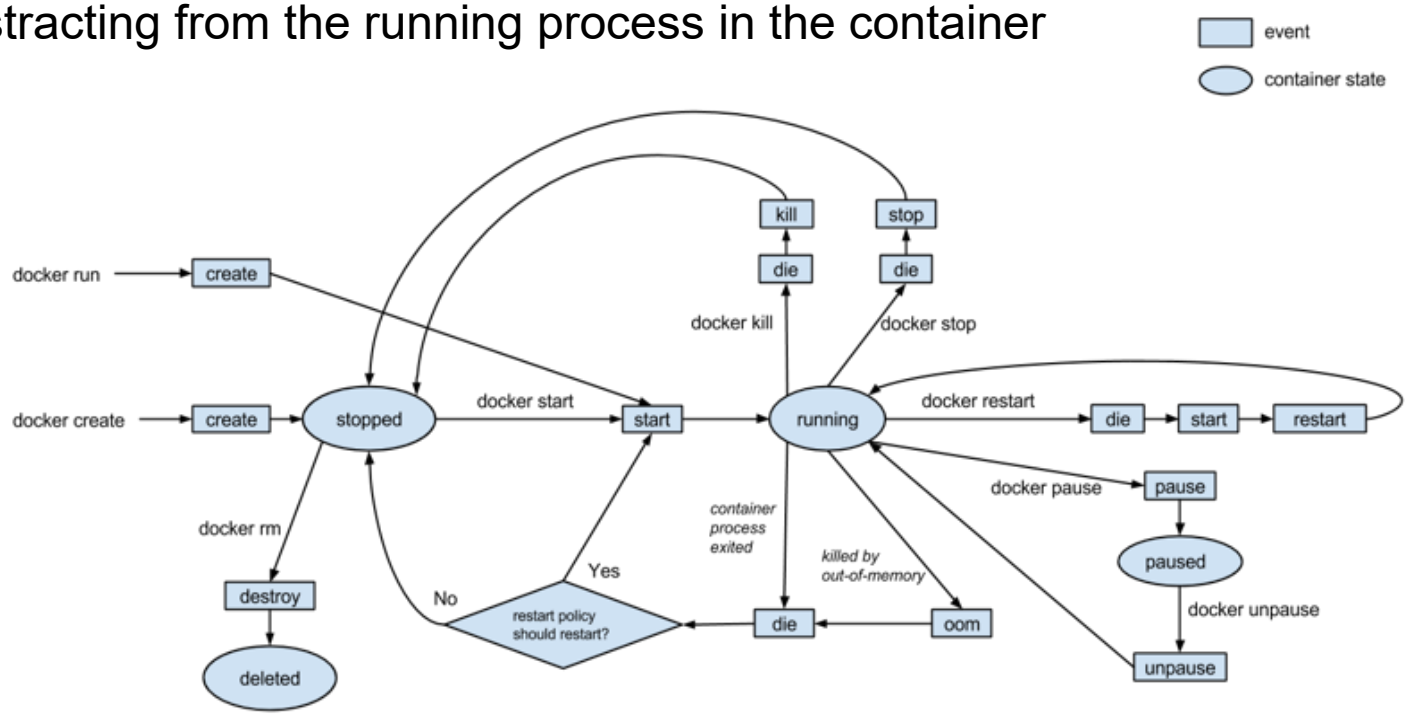


Client-server architecture

- Docker daemon
- API REST/Docker CLI

Docker container lifecycle events

Definition and handling of Events through **API**
abstracting from the running process in the container



Reproducible Environment

What we need?

- Hardware Platform
- Operating System
- OS Library
- Process Dependencies

When we need it?

- Development time
- Testing time
- Production

Dockerfile


Starting Image
Multi-Stage Build
syntax=docker/dockerfile:1
FROM golang:1.16
WORKDIR /go/src/github.com/alexellis/href-counter/
RUN go get -d -v golang.org/x/net/html
COPY app.go .
RUN CGO_ENABLED=0 GOOS=linux go build -a -installsuffix cgo -o app .
FROM alpine:latest
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=0 /go/src/github.com/alexellis/href-counter/app .
CMD ["./app"]
Configuration
NO Layer
creation
Modify fs
Layer creation

Docker Images

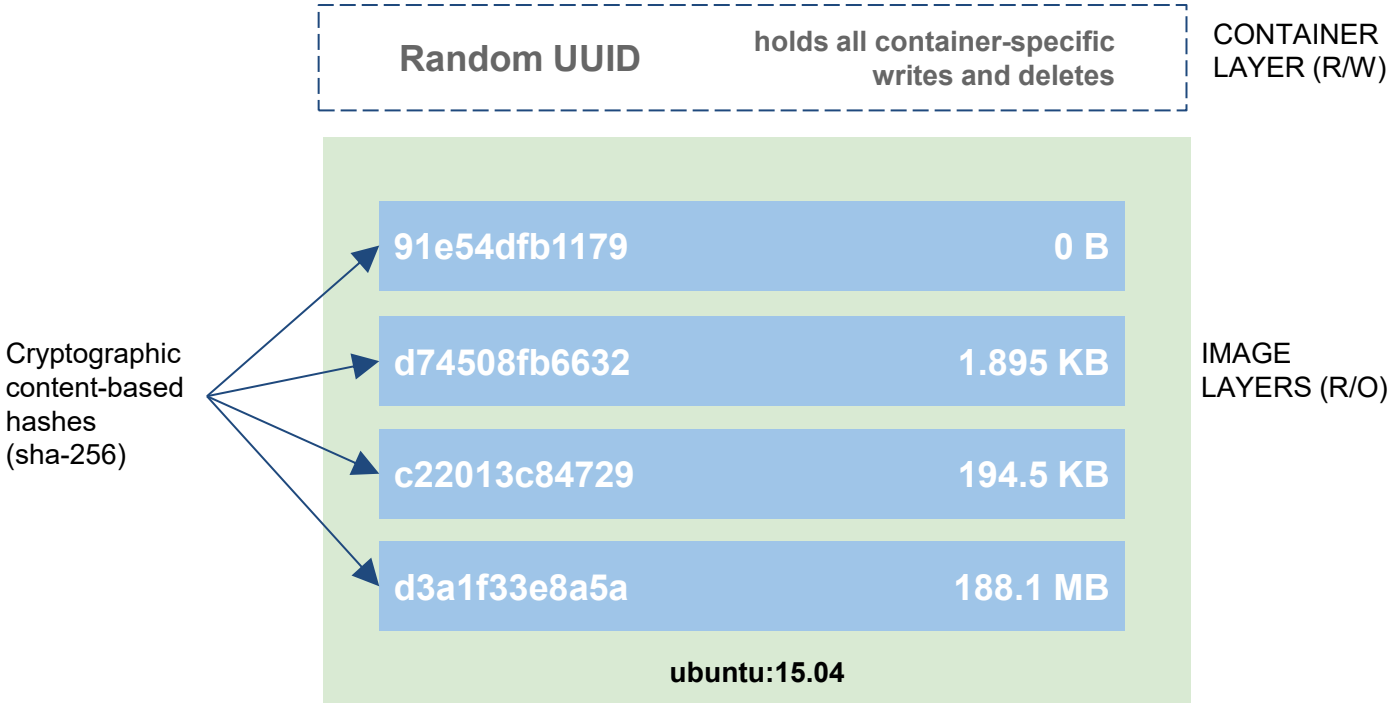
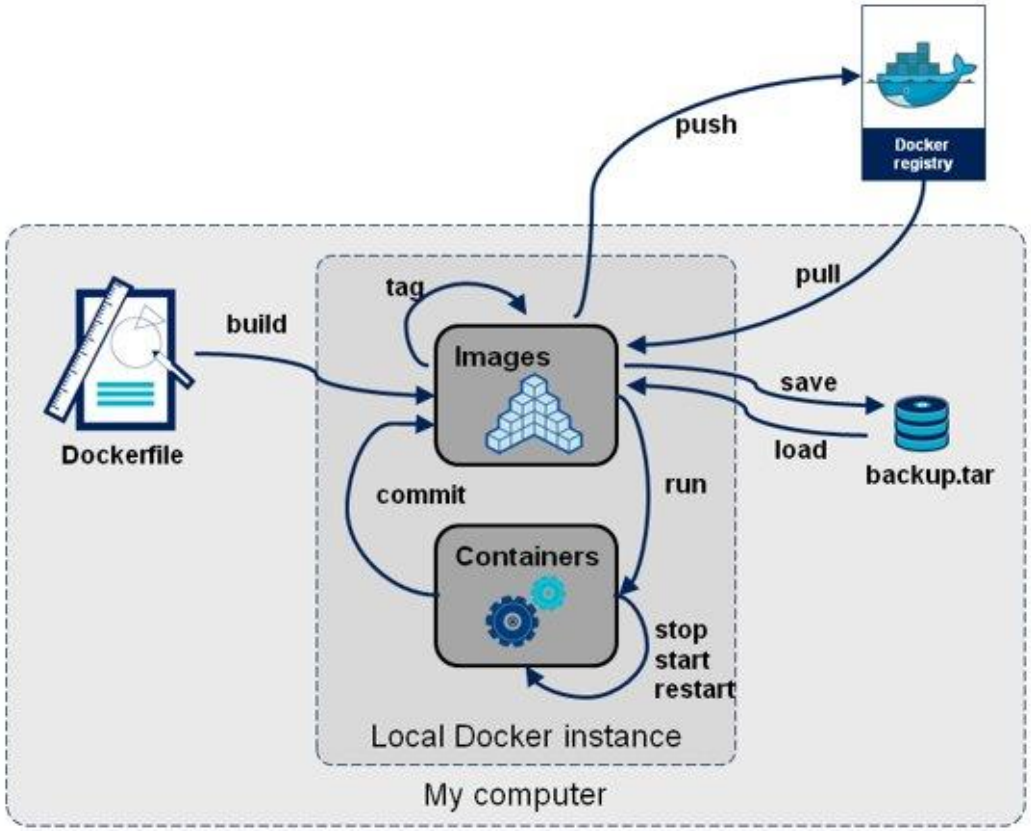
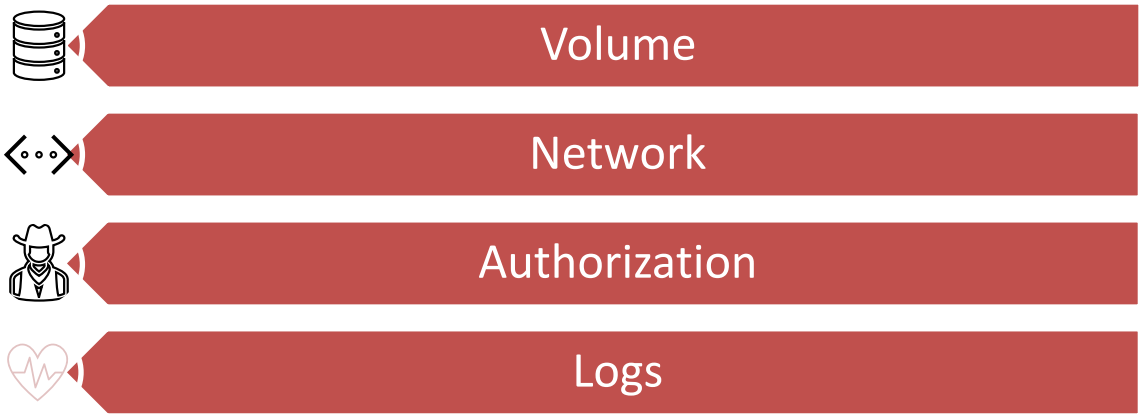


Image Lifecycle



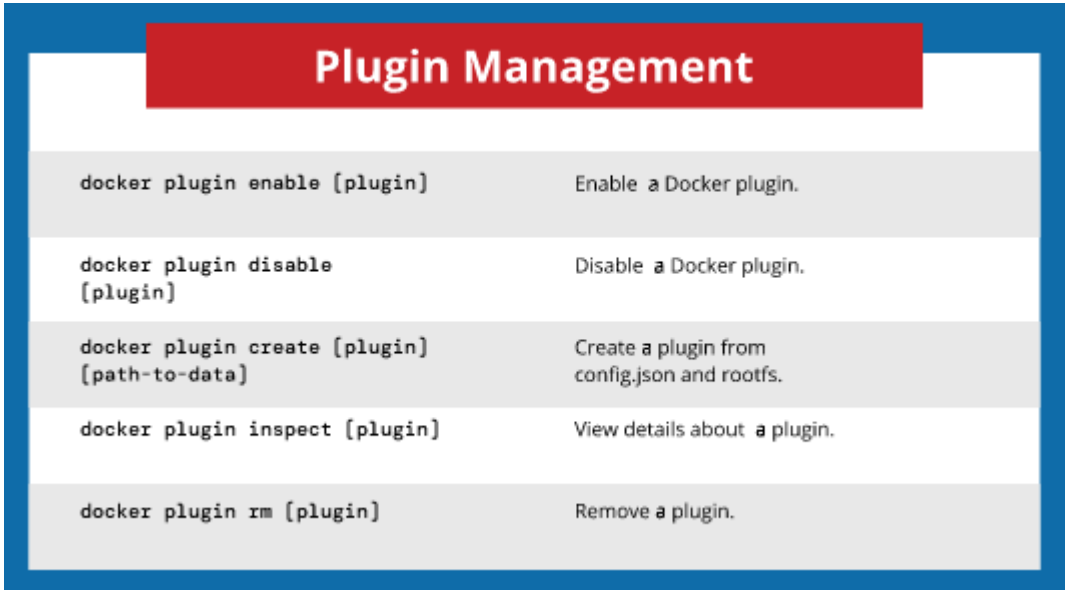
Docker plugins

Plugins extend Docker's functionality:

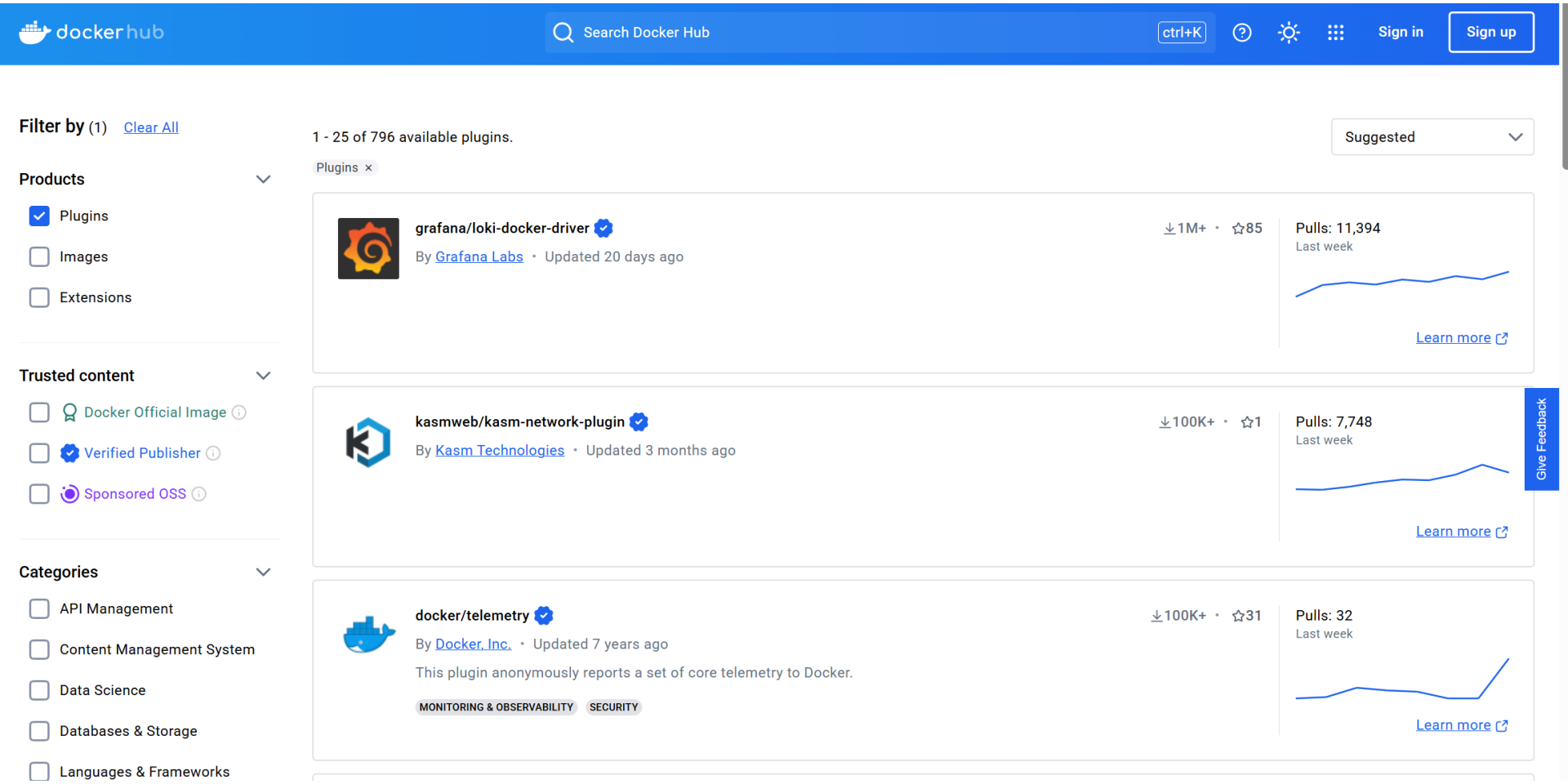


Plugins

Docker plugins extend the functionality of the Docker Engine by allowing third-party integrations for various tasks. These plugins come in different types, such as volume, network, and authorization plugins.
Volume plugins enable Docker **volumes** to persist across multiple hosts, while **network** plugins provide advanced network configurations and policies. **Authorization** plugins help manage access control within Docker environments. Installing a plugin typically involves following the specific instructions provided in the plugin's documentation. By leveraging these plugins, users can customize and enhance their Docker environments to better suit their needs



Plugins

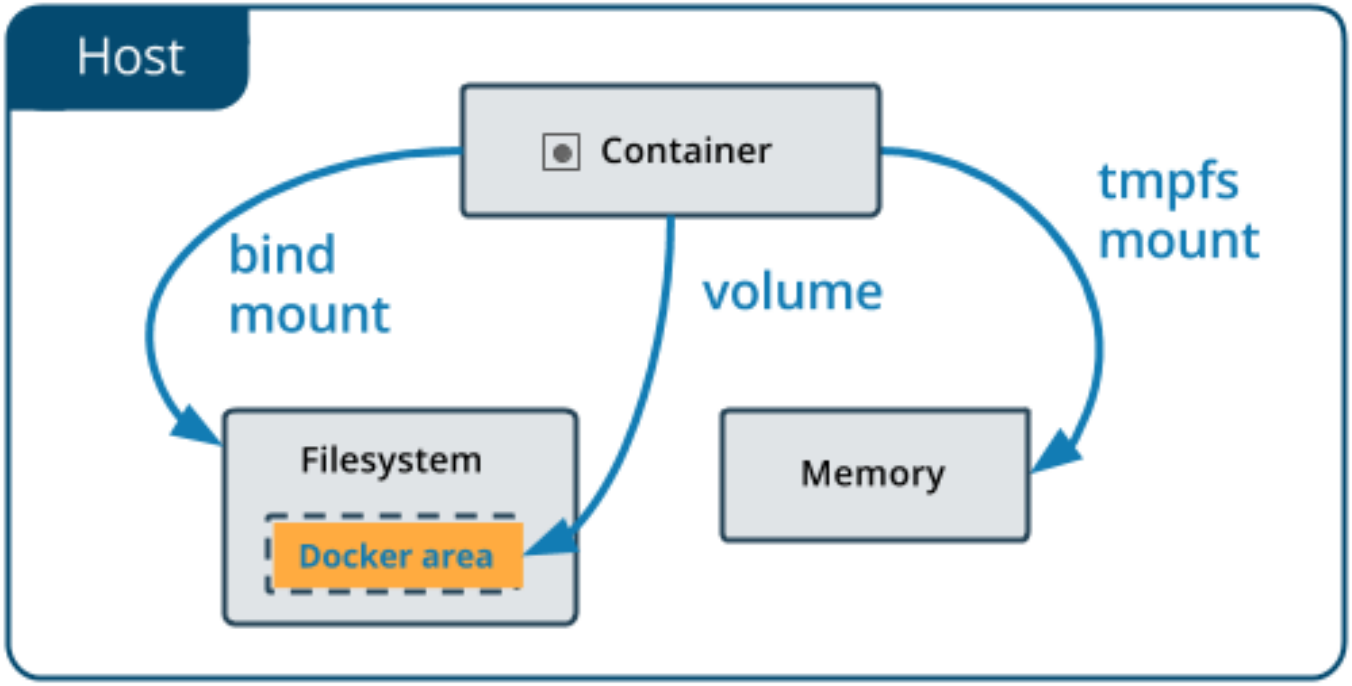


[Explore Docker's Container Image Repository | Docker Hub]

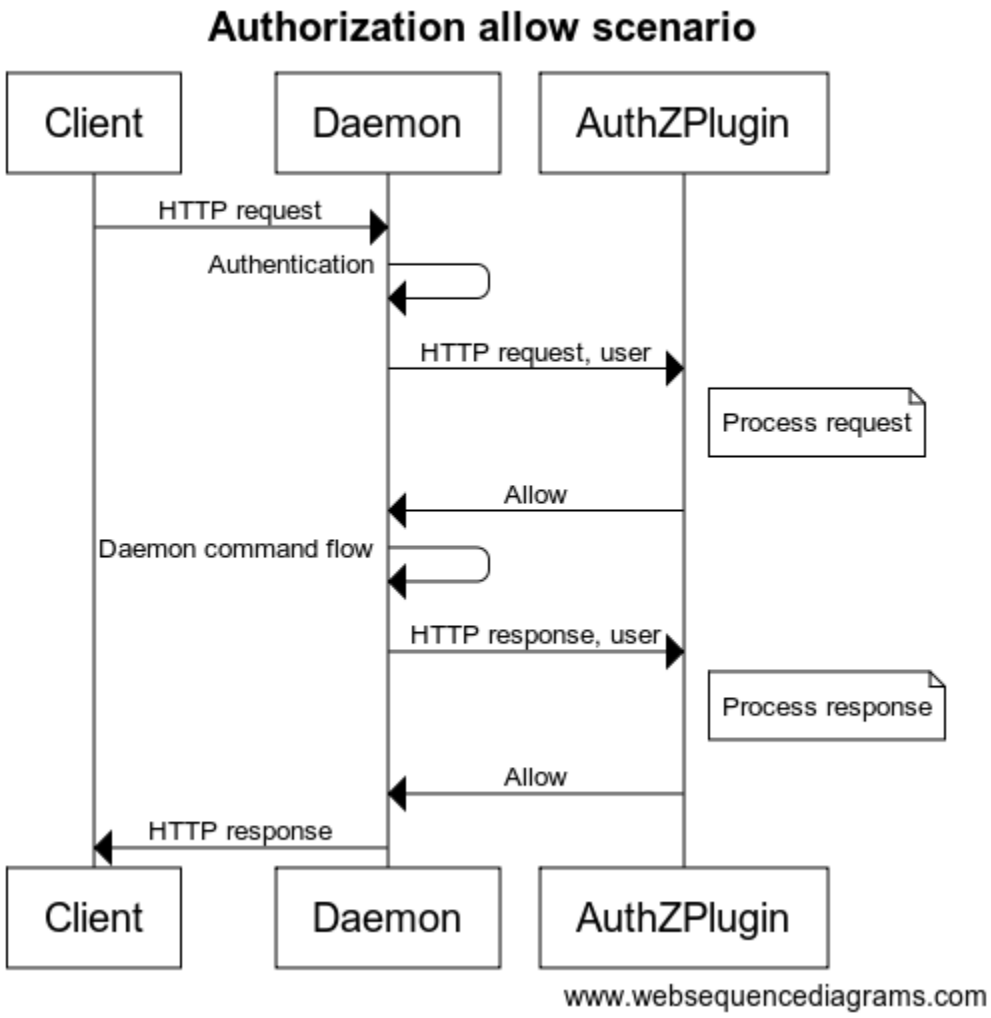
Networks

- **Bridge:** uses a software bridge which allows containers connected to the same bridge network to communicate, while providing isolation from containers which are not connected to that bridge network
- **Host:** Remove network isolation between the container and the Docker host, and use the host's networking directly
- **Overlay:** software network abstraction that can be used to run multiple separate, discrete virtualized network layers on top of the physical network (We will go deeper….)
- **Macvlan:** assign a MAC address to a container, making it appear as a physical device on your network. The Docker daemon routes traffic to containers by their MAC addresses
- **None:** disable all networking
- **External Plugin:** use third-party network plugins

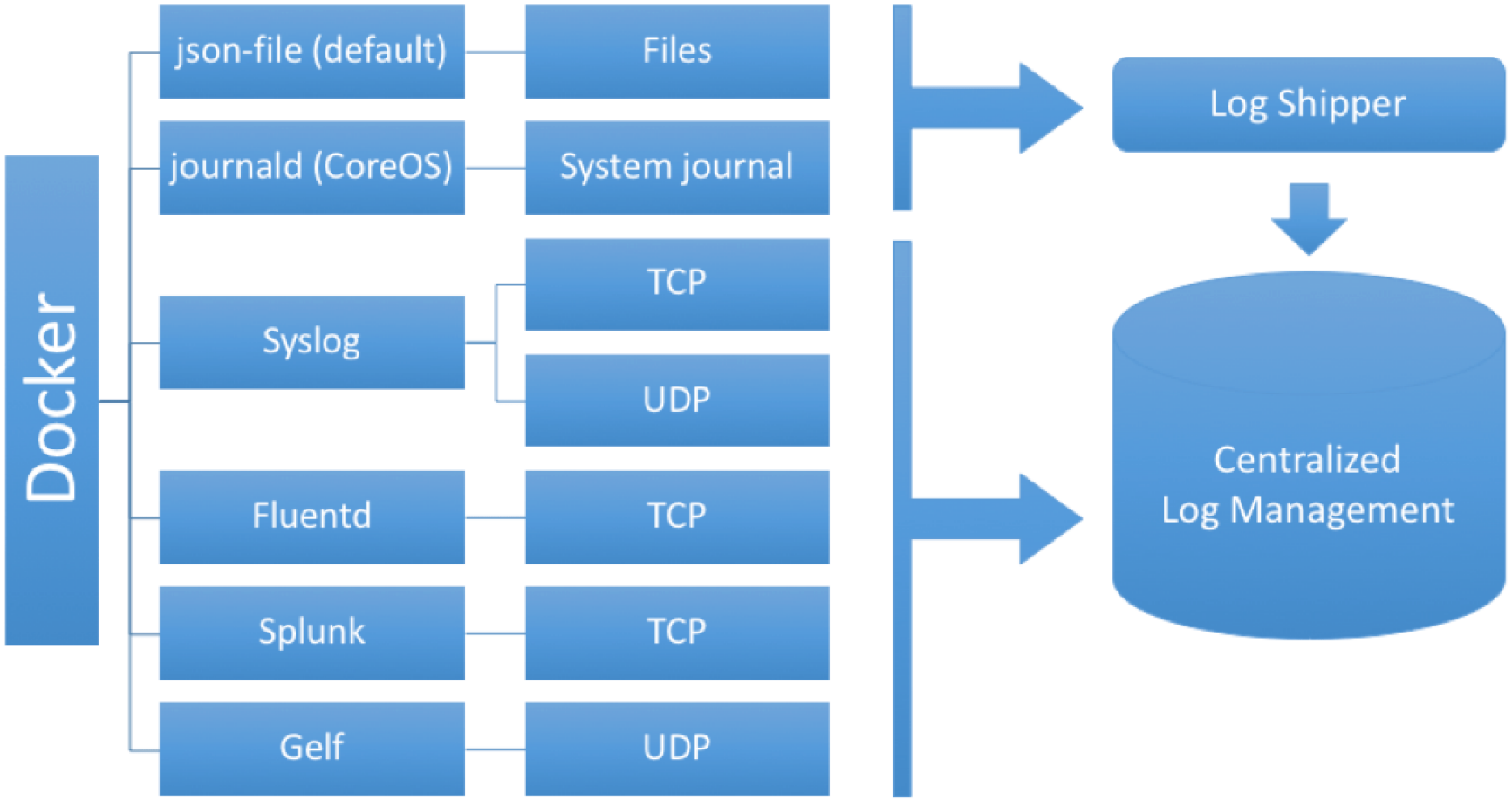
Persistence



Authorization



Logs


Docker
json-file (default)
Files
journald (CoreOS)
System journal
Syslog
TCP
UDP
Fluentd
TCP
Splunk
TCP
Gelf
UDP
Log Shipper
Centralized
Log Management

Configuration

Configure container execution

- Dockerfile
- Docker CLI
- Docker-Compose

Share configurations with the containerized process

- Docker secrets
- Docker Environment variables
- Docker Config files

Docker-Compose


```yaml
version: "3.9"  # optional since v1.27.0
services:
  web:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - .:/code
      - logvolume01:/var/log
    links:
      - redis
  redis:
    image: redis
volumes:
  logvolume01: {}
```

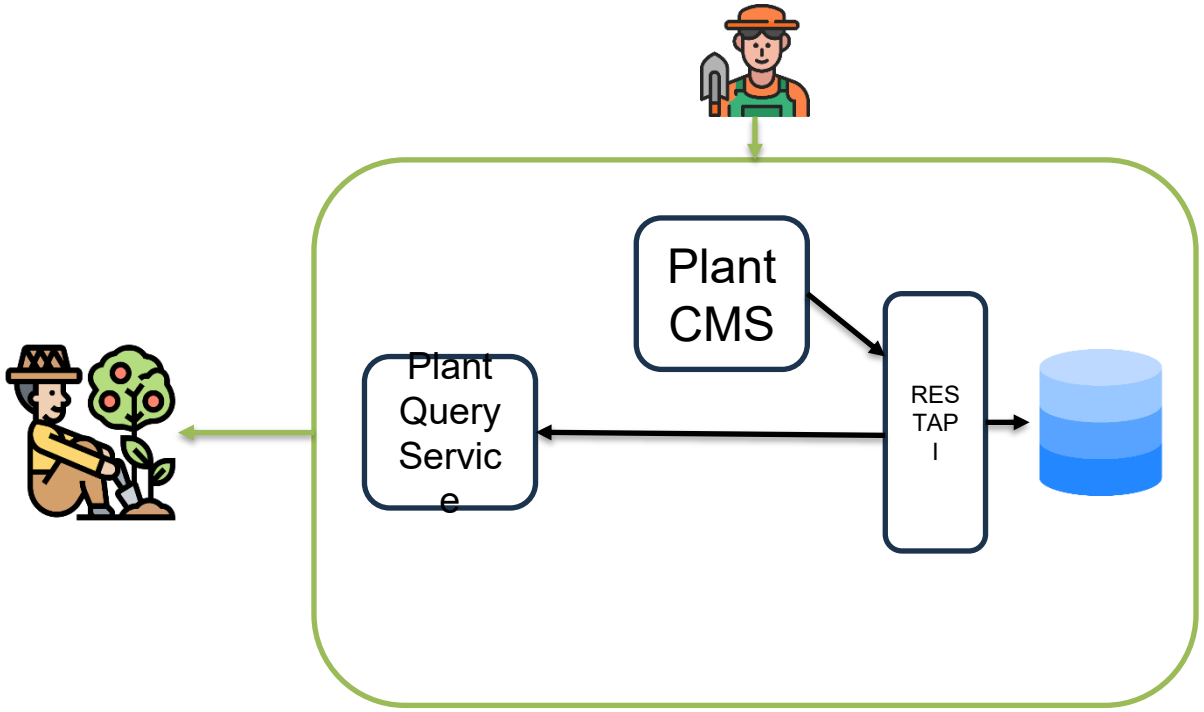
RECAP: Container vs VM vs Process

	Process	Container	VM
Definition	A representation of a running program.	Isolated group of processes managed by a shared kernel.	A full OS that shares host hardware via a hypervisor.
Use case	Abstraction to store state about a running process.	Creates isolated environments to run many apps.	Creates isolated environments to run many apps.
Type of OS	Same OS and distro as host,	Same kernel, but different distribution.	Multiple independent operating systems.
OS isolation	Memory space and user privilieges.	Namespaces and cgroups.	Full OS isolation.
Size	Whatever user's application uses.	Images measured in MB + user's application.	Images measured in GB + user's application.
Lifecycle	Created by forking, can be long or short lived, more often short.	Runs directly on kernel with no boot process, often is short lived.	Has a boot process and is typically long lived.

https://jessicagreben.medium.com/what-is-the-difference-between-a-process-a-container-and-a-vm-f36ba0f8a8f7

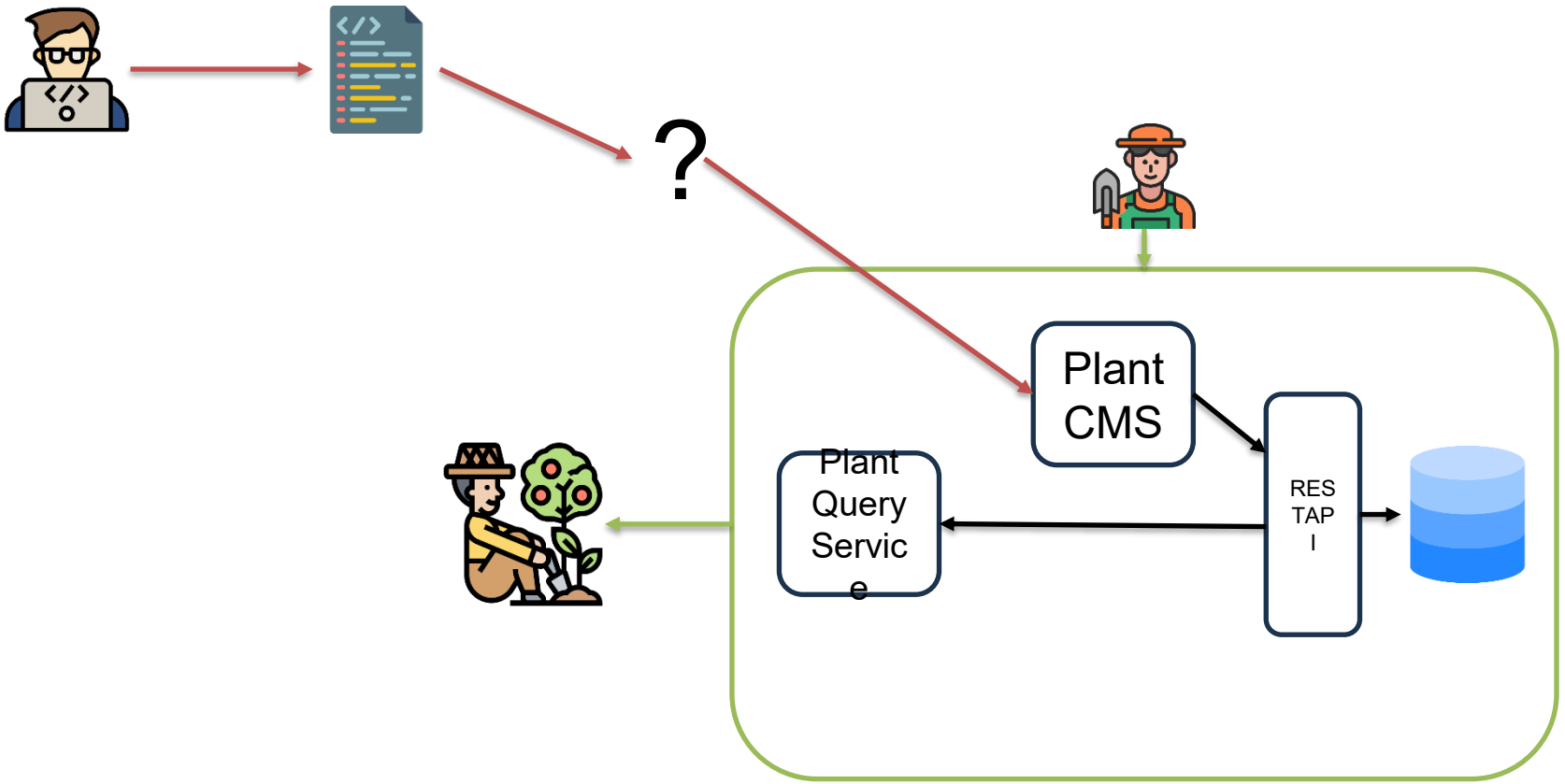
Plant App: Hosting

Question 1:
What can happen we host all the services in one single node?



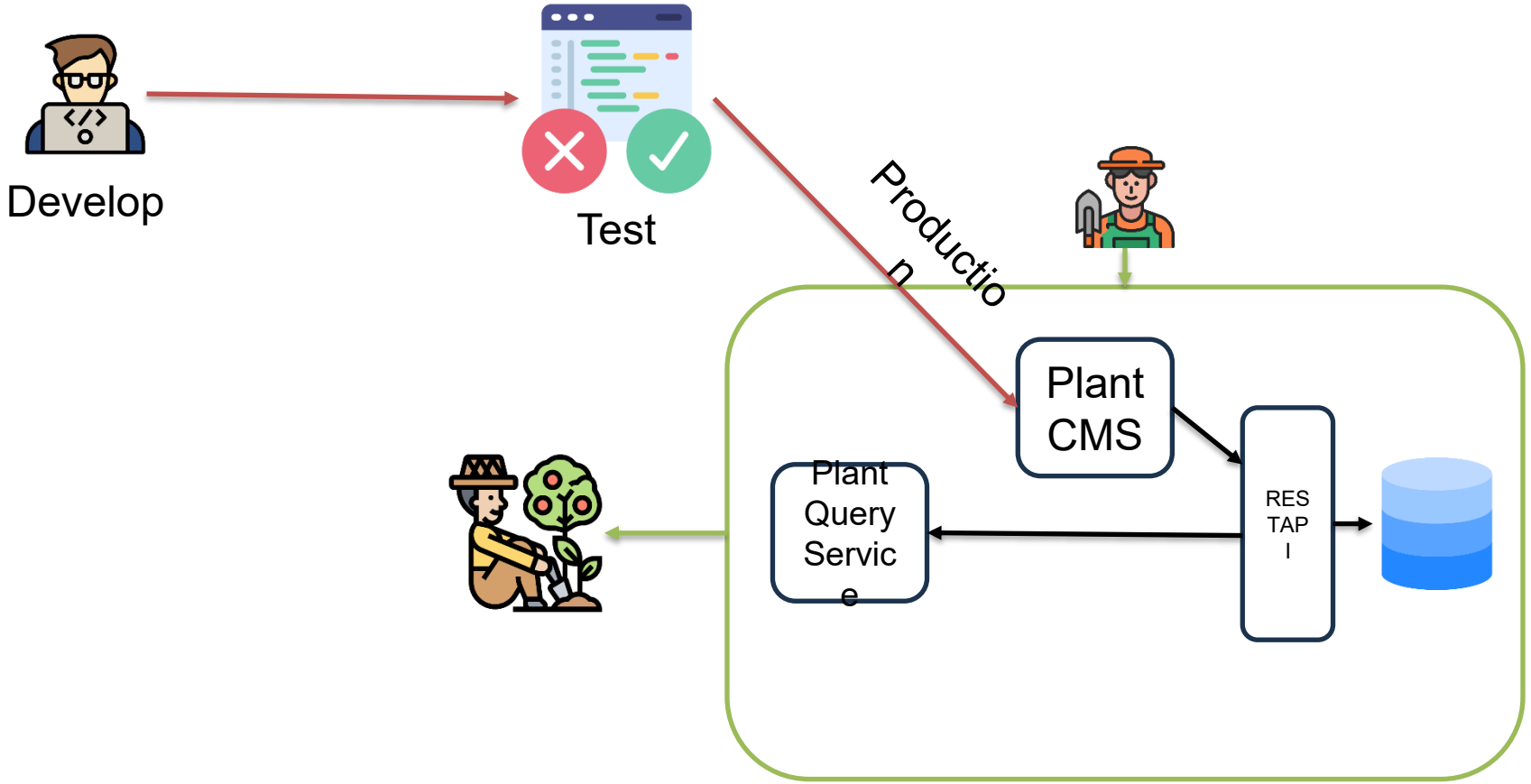
Plant App: Packing

Question 2:
How you can move new versions of the services to production?
What if you have to rollback to a precedent version?



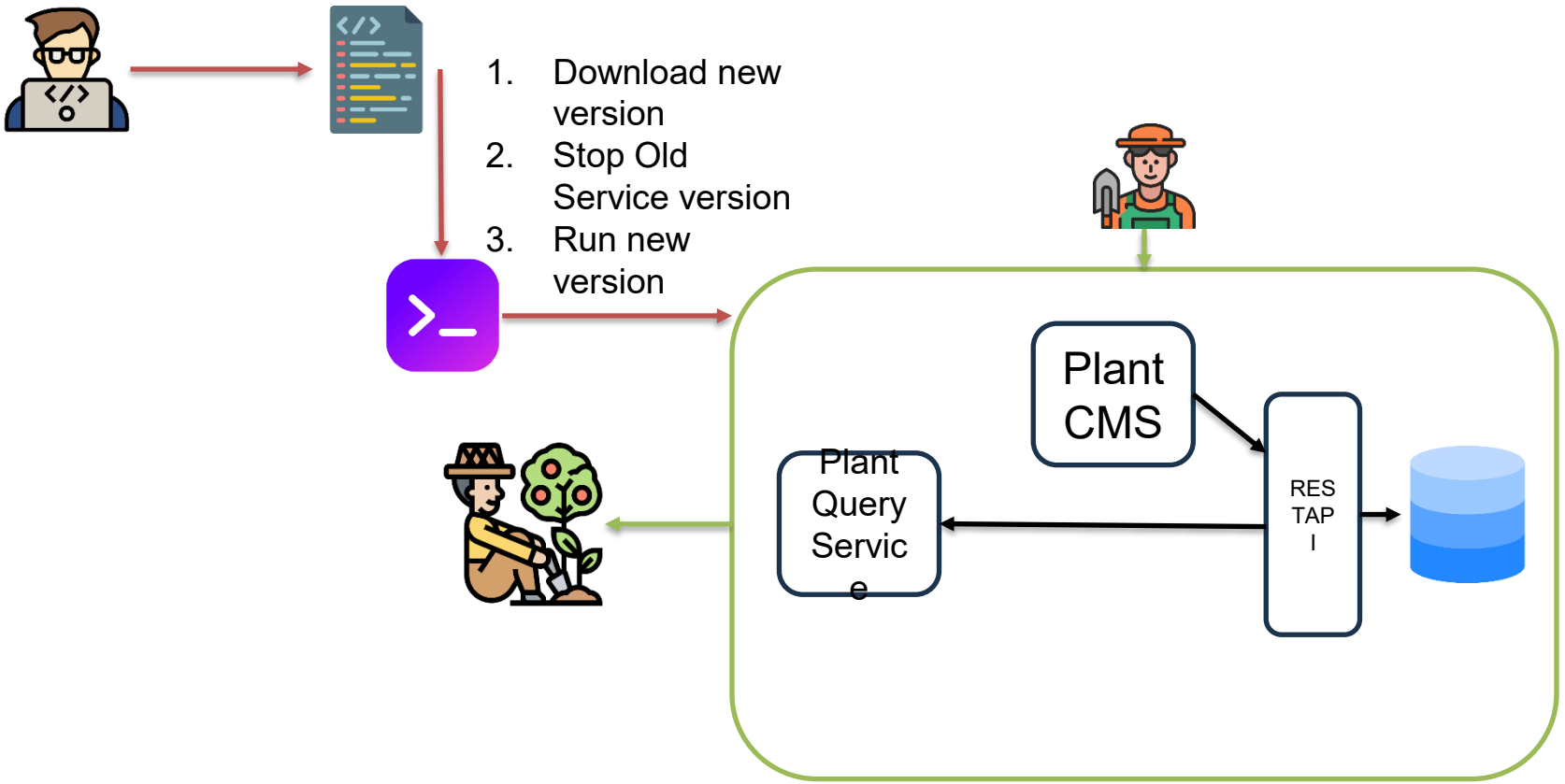
Plant App: Execution Environment

Question 3:
Our application will behave in the same way in your local pc, in a test environment, and in production?



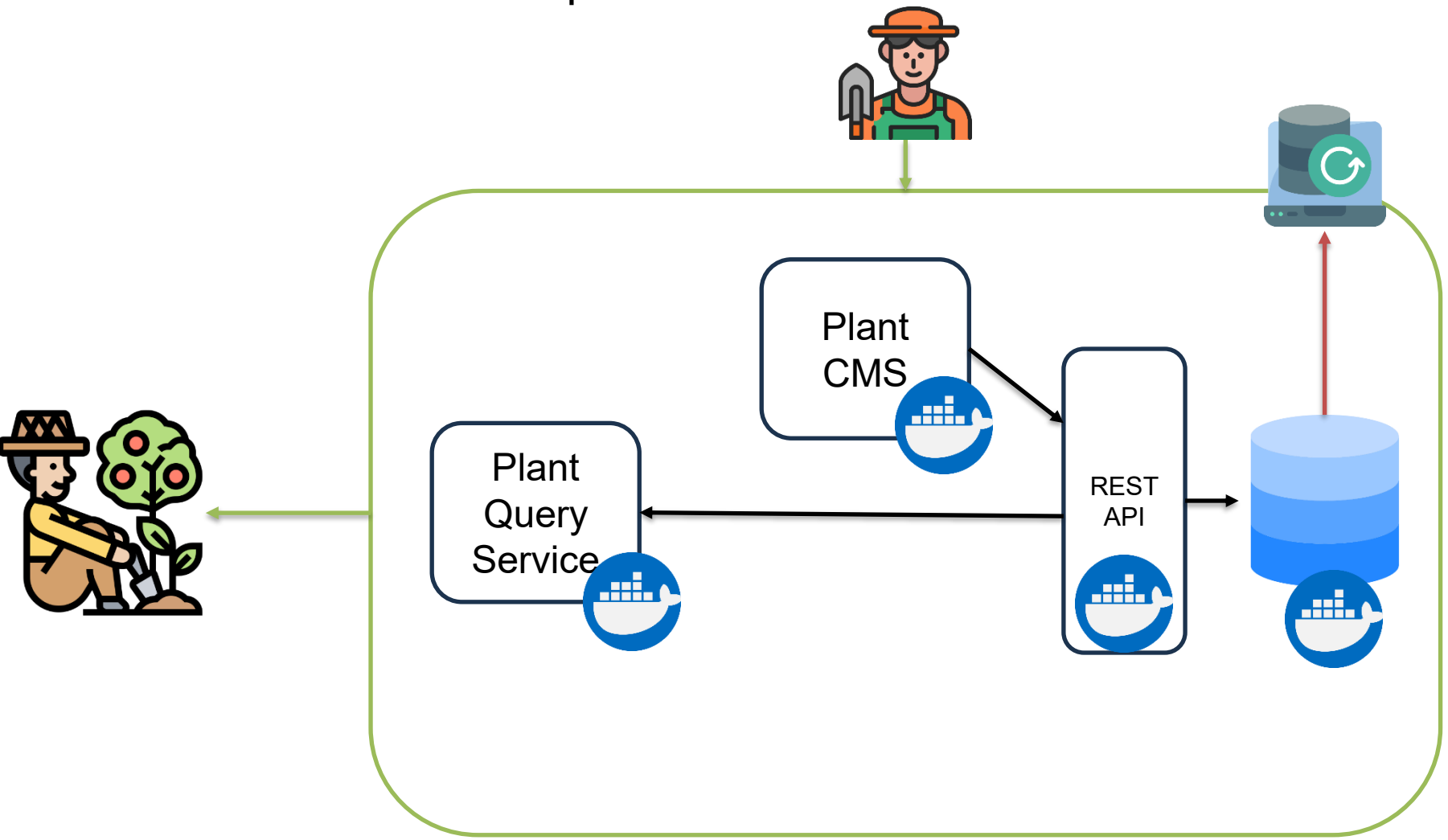
Plant App: Management

Question 4
How do you manage lifecycle of services?



Plant App: Exercise 1

Exercise1: Add a backup service



Short Project Activity

- Experiment with different containerization Technologies
  - WASM
  - Unikernel
  - Firecracker
  - LXC
- Create a custom Docker Plugin to support
  - Custom networking
  - Custom observability